Ex: 1

Date: 16/1/26

Study of Various network commands used in windows and Linux

1) arp

```
C:\Users\T490>arp

Displays and modifies the IP-to-Physical address translation tables used by
address resolution protocol (ARP).

ARP -s inet_addr eth_addr [if_addr]
ARP -d inet_addr [if_addr]
ARP -a [inet_addr] [-N if_addr] [-v]

  -a            Displays current ARP entries by interrogating the current
                protocol data.  If inet_addr is specified, the IP and Physical
                addresses for only the specified computer are displayed.  If
                more than one network interface uses ARP, entries for each ARP
                table are displayed.
  -g            Same as -a.
  -v            Displays current ARP entries in verbose mode.  All invalid
                entries and entries on the loop-back interface will be shown.
  inet_addr     Specifies an internet address.
  -N if_addr    Displays the ARP entries for the network interface specified
                by if_addr.
  -d            Deletes the host specified by inet_addr. inet_addr may be
                wildcarded with * to delete all hosts.
  -s            Adds the host and associates the Internet address inet_addr
                with the Physical address eth_addr.  The Physical address is
                given as 6 hexadecimal bytes separated by hyphens. The entry
                is permanent.
  eth_addr      Specifies a physical address.
  if_addr       If present, this specifies the Internet address of the
                interface whose address translation table should be modified.
                If not present, the first applicable interface will be used.

Example:
  > arp -s 157.55.85.212   00-aa-00-62-c6-09  .... Adds a static entry.
  > arp -a                                    .... Displays the arp table.
```

2) hostname

```
C:\Users\T490>hostname
DESKTOP-8D4CFVT
```

3) ip config

```
C:\Users\T490>ipconfig

Windows IP Configuration


Ethernet adapter Ethernet:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Link-local IPv6 Address . . . . . : fe80::4dce:b7cb:d52a:d327%19
   IPv4 Address. . . . . . . . . . . : 192.168.56.1
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :

Wireless LAN adapter Local Area Connection* 1:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Local Area Connection* 10:

   Media State . . . . . . . . . . . : Media disconnected
   Connection-specific DNS Suffix  . :

Wireless LAN adapter Wi-Fi:

   Connection-specific DNS Suffix  . : comnect.local
   Link-local IPv6 Address . . . . . : fe80::75c2:a26b:51cc:50c8%9
   IPv4 Address. . . . . . . . . . . : 192.168.1.2
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . : fe80::1%9
                                       192.168.1.1
```

4) nbtstat

```
C:\Users\T490>nbtstat

Displays protocol statistics and current TCP/IP connections using NBT
(NetBIOS over TCP/IP).

NBTSTAT [ [-a RemoteName] [-A IP address] [-c] [-n]
        [-r] [-R] [-RR] [-s] [-S] [interval] ]

  -a   (adapter status) Lists the remote machine's name table given its name
  -A   (Adapter status) Lists the remote machine's name table given its
                        IP address.
  -c   (cache)          Lists NBT's cache of remote [machine] names and their IP addresses
  -n   (names)          Lists local NetBIOS names.
  -r   (resolved)       Lists names resolved by broadcast and via WINS
  -R   (Reload)         Purges and reloads the remote cache name table
  -S   (Sessions)       Lists sessions table with the destination IP addresses
  -s   (sessions)       Lists sessions table converting destination IP
                        addresses to computer NETBIOS names.
  -RR  (ReleaseRefresh) Sends Name Release packets to WINS and then, starts Refresh

  RemoteName   Remote host machine name.
  IP address   Dotted decimal representation of the IP address.
  interval     Redisplays selected statistics, pausing interval seconds
               between each display. Press Ctrl+C to stop redisplaying
               statistics.
```

5) netstat

```
C:\Users\T490>netstat

Active Connections

  Proto  Local Address          Foreign Address        State
  TCP    192.168.1.2:49702      4.213.25.242:https     ESTABLISHED
  TCP    192.168.1.2:49736      104.26.9.75:https      ESTABLISHED
  TCP    192.168.1.2:49773      4.213.25.242:https     ESTABLISHED
  TCP    192.168.1.2:49859      sf-in-f188:5228        ESTABLISHED
  TCP    192.168.1.2:49973      104.16.249.249:https   ESTABLISHED
  TCP    192.168.1.2:49989      lcmaaa-ax-in-f5:https  ESTABLISHED
  TCP    192.168.1.2:50009      static:https           CLOSE_WAIT
  TCP    192.168.1.2:50012      20.190.145.141:https   TIME_WAIT
```

6) pathping

```
C:\Users\T490>pathping

Usage: pathping [-g host-list] [-h maximum_hops] [-i address] [-n]
                [-p period] [-q num_queries] [-w timeout]
                [-4] [-6] target_name

Options:
    -g host-list     Loose source route along host-list.
    -h maximum_hops  Maximum number of hops to search for target.
    -i address       Use the specified source address.
    -n               Do not resolve addresses to hostnames.
    -p period        Wait period milliseconds between pings.
    -q num_queries   Number of queries per hop.
    -w timeout       Wait timeout milliseconds for each reply.
    -4               Force using IPv4.
    -6               Force using IPv6.
```

7) nslookup

```
C:\Users\T490>nslookup
Default Server:  RTK_GW.comnect.local
Address:  192.168.1.1

> exit

C:\Users\T490>nslookup www.google.com
Server:  RTK_GW.comnect.local
Address:  192.168.1.1

Non-authoritative answer:
Name:    www.google.com
Addresses:  2001:4860:4826:7700::
          2001:4860:4827:7700::
          2001:4860:4828:7700::
          2001:4860:4829:7700::
          2001:4860:482c:7700::
          2001:4860:482d:7700::
          2001:4860:482a:7700::
          2001:4860:482b:7700::
          142.251.156.119
          142.251.155.119
          142.251.150.119
          142.251.157.119
          142.251.154.119
          142.251.153.119
          142.251.152.119
          142.251.151.119
```

8) ping

```
C:\Users\T490>ping

Usage: ping [-t] [-a] [-n count] [-l size] [-f] [-i TTL] [-v TOS]
            [-r count] [-s count] [[-j host-list] | [-k host-list]]
            [-w timeout] [-R] [-S srcaddr] [-c compartment] [-p]
            [-4] [-6] target_name

Options:
    -t             Ping the specified host until stopped.
                   To see statistics and continue - type Control-Break;
                   To stop - type Control-C.
    -a             Resolve addresses to hostnames.
    -n count       Number of echo requests to send.
    -l size        Send buffer size.
    -f             Set Don't Fragment flag in packet (IPv4-only).
    -i TTL         Time To Live.
    -v TOS         Type Of Service (IPv4-only. This setting has been deprecated
                   and has no effect on the type of service field in the IP
                   Header).
    -r count       Record route for count hops (IPv4-only).
    -s count       Timestamp for count hops (IPv4-only).
    -j host-list   Loose source route along host-list (IPv4-only).
    -k host-list   Strict source route along host-list (IPv4-only).
    -w timeout     Timeout in milliseconds to wait for each reply.
    -R             Use routing header to test reverse route also (IPv6-only).
                   Per RFC 5095 the use of this routing header has been
                   deprecated. Some systems may drop echo requests if
                   this header is used.
    -S srcaddr     Source address to use.
    -c compartment Routing compartment identifier.
    -p             Ping a Hyper-V Network Virtualization provider address.
    -4             Force using IPv4.
    -6             Force using IPv6.
```

9) Route

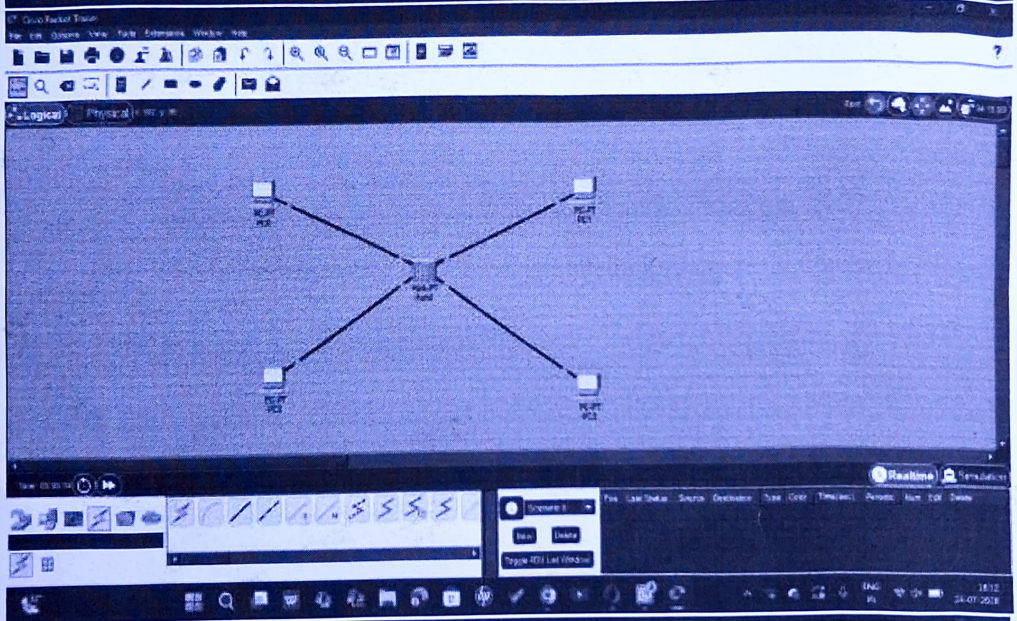
```
C:\Users\T490>route

Manipulates network routing tables.

ROUTE [-f] [-p] [-4|-6] command [destination]
                  [MASK netmask]  [gateway] [METRIC metric]  [IF interface]

  -f            Clears the routing tables of all gateway entries.  If this is
                used in conjunction with one of the commands, the tables are
                cleared prior to running the command.

  -p            When used with the ADD command, makes a route persistent across
                boots of the system. By default, routes are not preserved
                when the system is restarted. Ignored for all other commands,
                which always affect the appropriate persistent routes.

  -4            Force using IPv4.

  -6            Force using IPv6.

  command       One of these:
                  PRINT     Prints  a route
                  ADD       Adds    a route
                  DELETE    Deletes a route
                  CHANGE    Modifies an existing route
  destination   Specifies the host.
  MASK          Specifies that the next parameter is the 'netmask' value.
  netmask       Specifies a subnet mask value for this route entry.
                If not specified, it defaults to 255.255.255.255.
  gateway       Specifies gateway.
  interface     the interface number for the specified route.
  METRIC        specifies the metric, ie. cost for the destination.

All symbolic names used for destination are looked up in the network database
file NETWORKS. The symbolic names for gateway are looked up in the host name
database file HOSTS.

If the command is PRINT or DELETE. Destination or gateway can be a wildcard,
(wildcard is specified as a star '*'), or the gateway argument may be omitted.
```

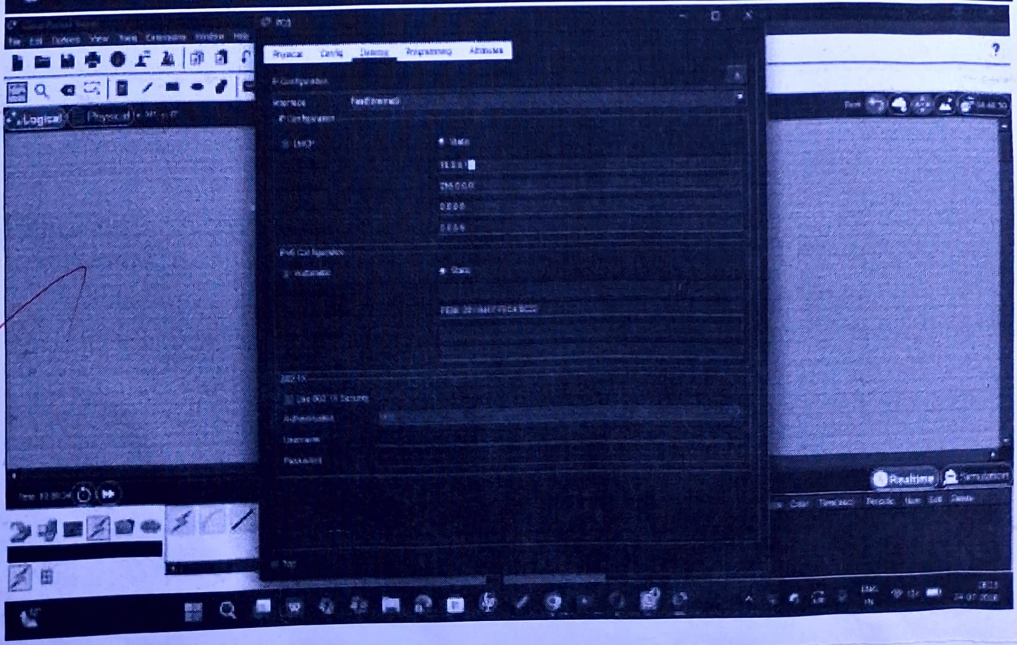
```
If Dest contains a * or ?, it is treated as a shell pattern, and only
matching destination routes are printed. The '*' matches any string,
and '?' matches any one char. Examples: 157.*.1, 157.*, 127.*, *224*.

Pattern match is only allowed in PRINT command.
Diagnostic Notes:
    Invalid MASK generates an error, that is when (DEST & MASK) != DEST.
    Example> route ADD 157.0.0.0 MASK 155.0.0.0 157.55.80.1 IF 1
             The route addition failed: The specified mask parameter is invalid. (Destination & Mask) != Destination.

Examples:

    > route PRINT
    > route PRINT -4
    > route PRINT -6
    > route PRINT 157*            .... Only prints those matching 157*

    > route ADD 157.0.0.0 MASK 255.0.0.0  157.55.80.1 METRIC 3 IF 2
            destination^      ^mask      ^gateway     metric^    ^
                                                      Interface^
      If IF is not given, it tries to find the best interface for a given
      gateway.
    > route ADD 3ffe::/32 3ffe::1

    > route CHANGE 157.0.0.0 MASK 255.0.0.0 157.55.80.5 METRIC 2 IF 2

      CHANGE is used to modify gateway and/or metric only.

    > route DELETE 157.0.0.0
    > route DELETE 3ffe::/32
```

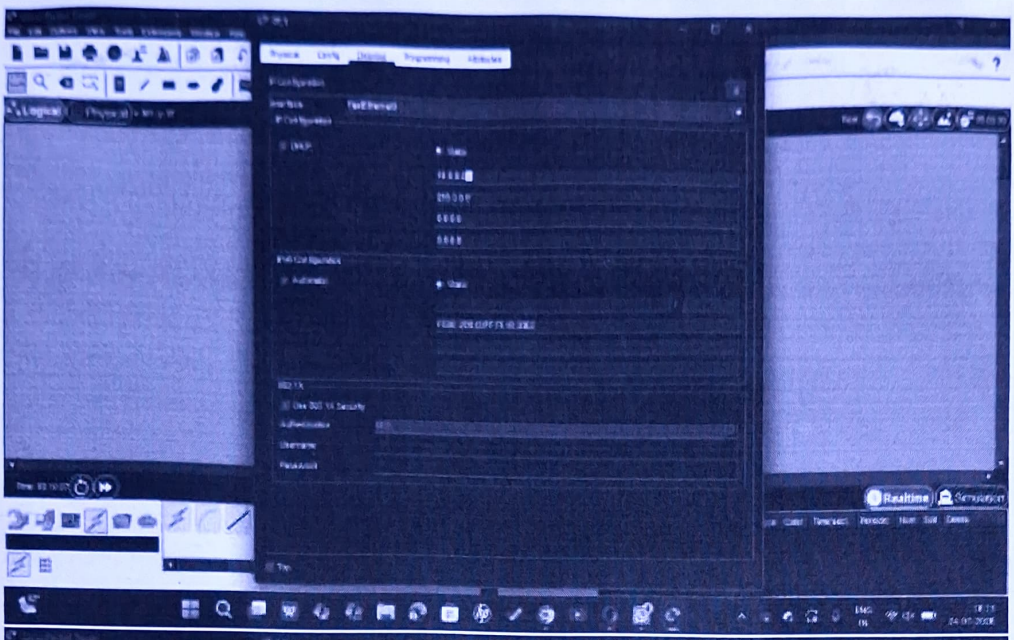
Exp no: 3

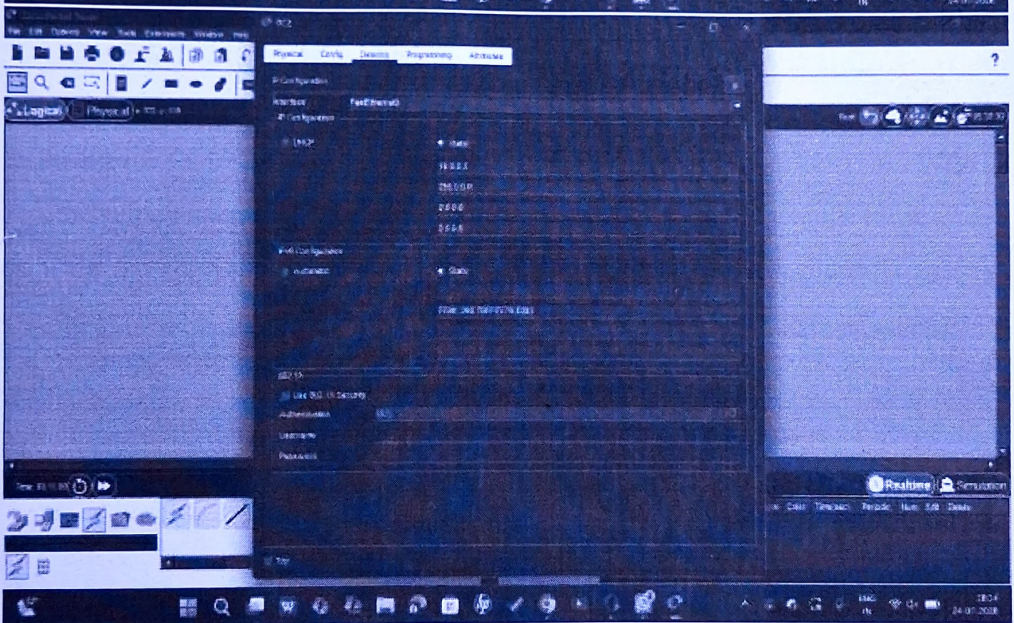
Date: 25/7/26

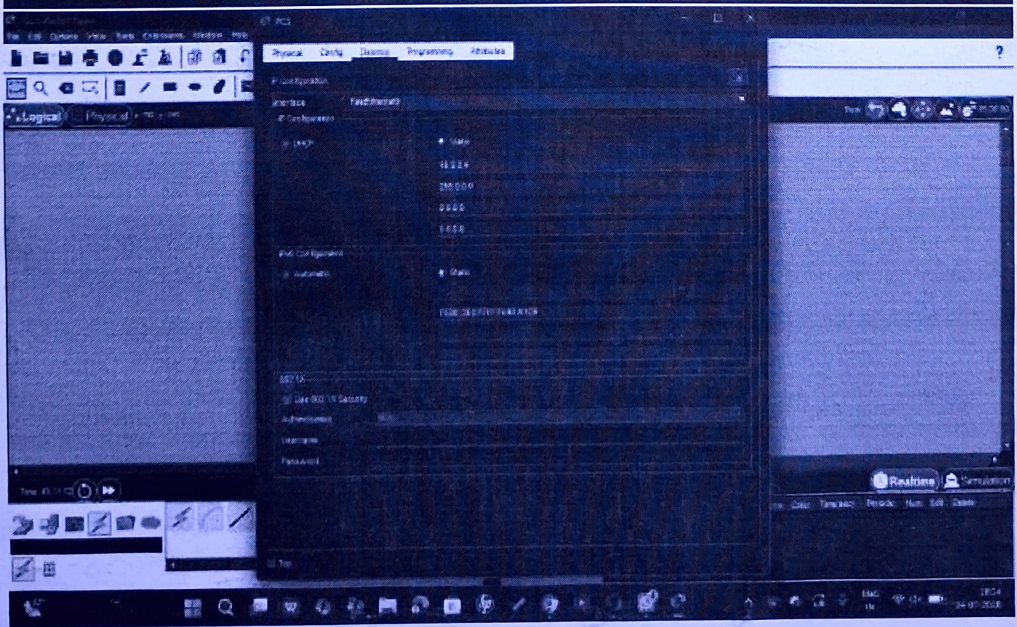
Experiments on CISCO PACKET TRACKER (Simulation tool)

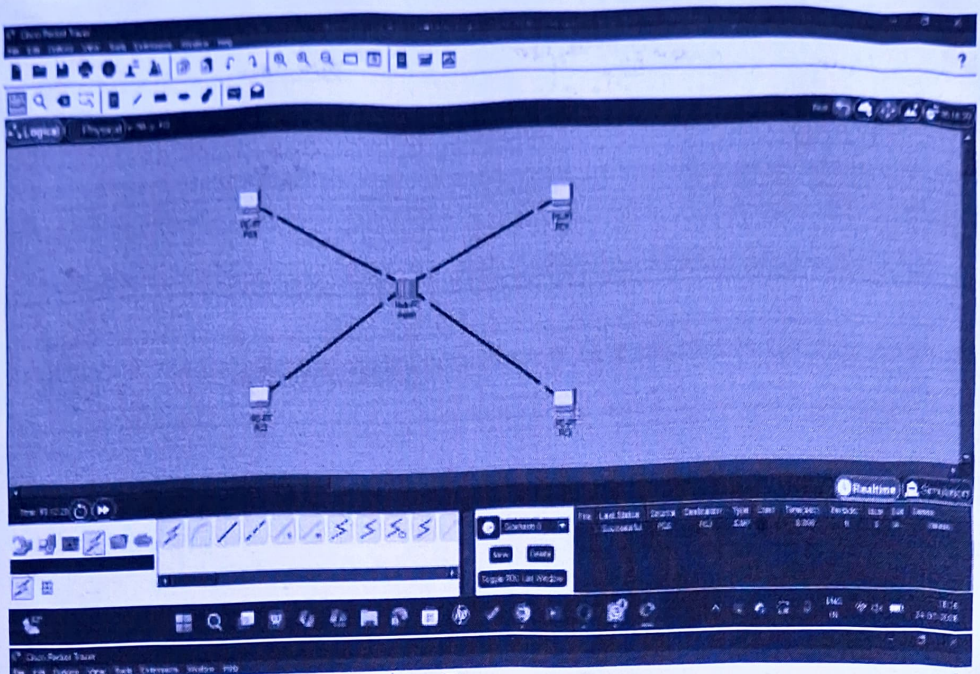


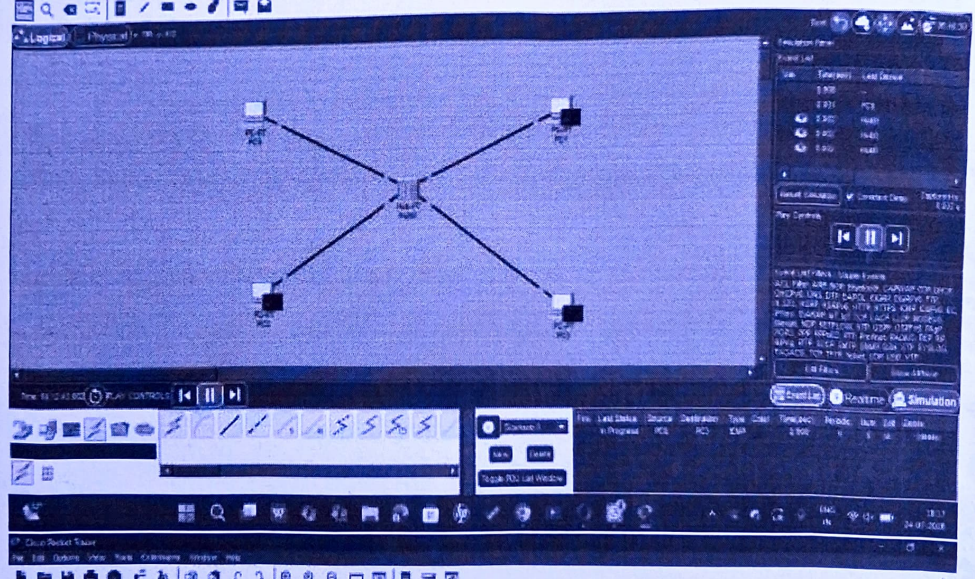


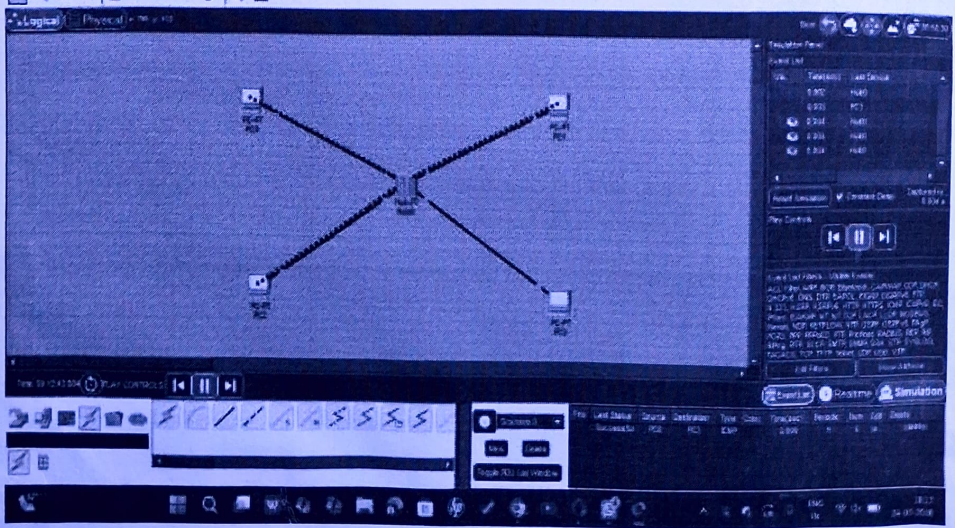


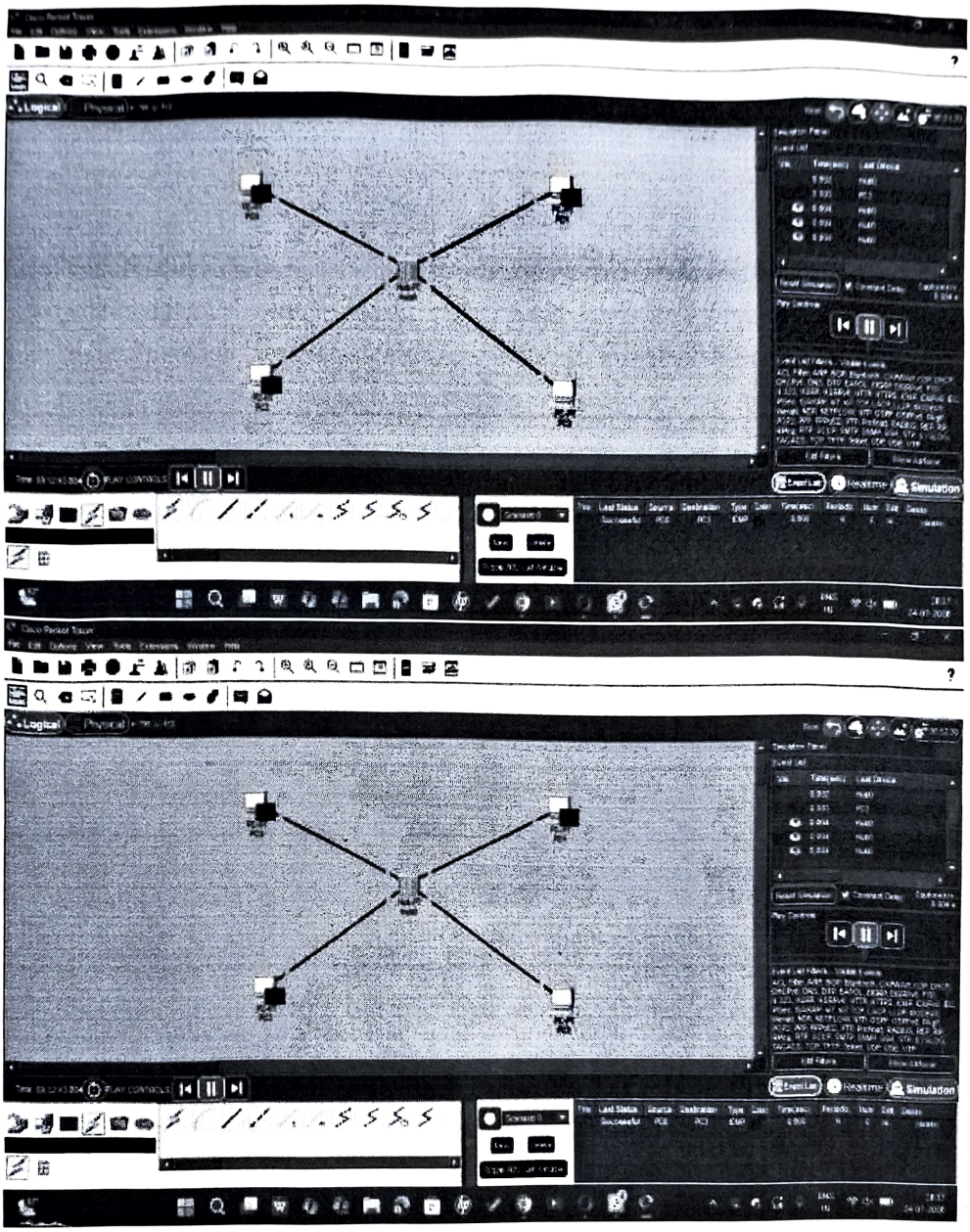










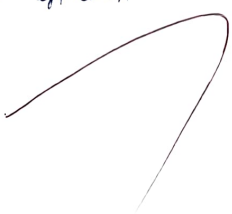


Procedure:

* Downloaded & installed cisco Tracker 9.0.0
* opened cisco packet tracker & used logical workspace.

* Created three networks
  - => one using HUB
  - => one using switch
  - => One using both hub & switch.
* Connected 4 pcs in each network using copper straight cables
* Assigned ip addresses to all pcs
* Sent PDU from one pc to another to test communication

HUB - sends data to all devices (broadcast), causing more traffic.
switch - sends data only to the correct twice, making it faster and efficient.



Result:

Successfully studied cisco packet tracker, created HUB, switch and both based networks, configured ip addresses, and tested communication.

Exp: 2

Date: 26/7/24

Study of different types of network cables

a) Understand different types of network cable

Different type of cables used in networking are.

1) Unshielded Twisted pair (UTP) cable.
2) Shielded Twisted pair (STP) cable
3) coaxial cable
4) Fibre optic cable.

Cable type	Category	max data transmission	Advantages / Disadvantages	Application / Use
UTP	category 3	10 bps	Advantages: • cheaper in cost • Easy to install as they have a smaller overall dia. Disadvantages: • More prone to (EMI) electromagnetic interference and noise	10 base-T ethernet fast ethernet, gigabit ethernet fast ethernet, Gigabit ethernet.
	Category 5	up to 100 mbps		
	category 5e	1 Gbps		
STP	Category 6, 6a	10 Gbps	advantages: • shielded • faster than UTP. • Less suspectable to noise and interference. Disadvantages: • Expensive • Greater installation effort	Gigabit ethernet, 10G ethernet (55m) widely used in data centers.
SSTP	category 7	10 Gbps		Gigabit ethernet, 10G Ethernet (100m)
Coaxial cable	RG-6 RG-59 RG-11	10-100mbps	Advantages: • High bandwidth • Immune to interference • Low loss bandwidth • Versatile Disadvantages: • Limited distance • Cost • Size is bulky	Speed of signal is 500m. TV network High speed Internet connections.

Fibre Optics cable	Single mode multimode	100 Gbps	Advantages: • High speed • High bandwidth • High security • Long distance Disadvantages: • Expensive • Requires skilled Installers	maximum distance of fibre optic cable is around 100 meters.

b) make you own ethernet cross-over cable / straight cable:

Tools and parts needed:

- Ethernet cabling, CAT 5e is certified for gigabit support, but CAT 5 cabling works as well, just over shorter distances.
- A crimping tool. This is an all-in-one networking tool shaped to push down the pins in the plug and strip and cut the shielding off the cables.
- Two RJ 45 plugs
- Optional ~~two~~ play shields.

Step 1:

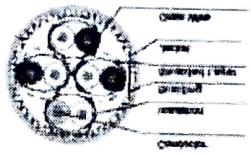
To start construction of the device, begin by threading shields onto the cable.

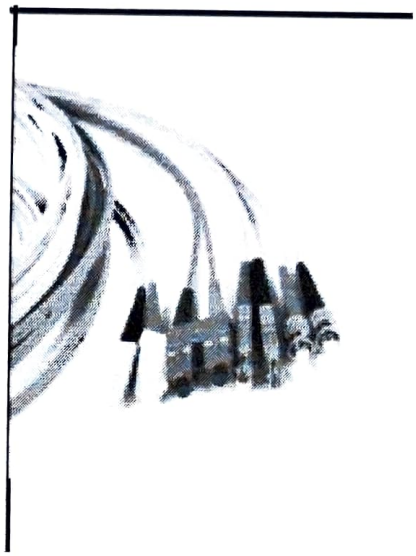
Step 2:

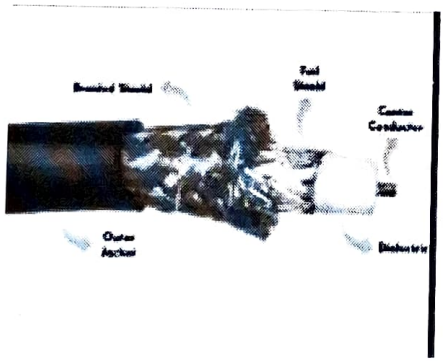
Next, strip approximately 1.5 cm of cable shielding from both ends. The crimping tool has a round area to complete this task.

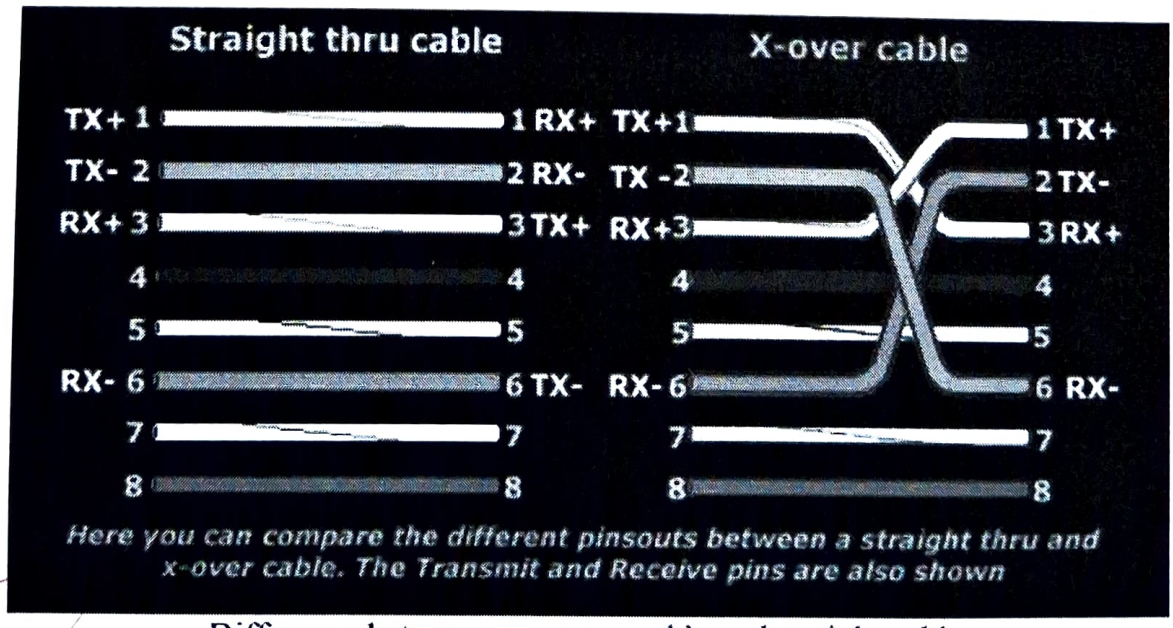
Step 3:

After, you will need to untangle the wires, there should be four "twisted pairs". Referring back to the sheet, arrange them from top to bottom. One end should be in arrangement A and other in B.



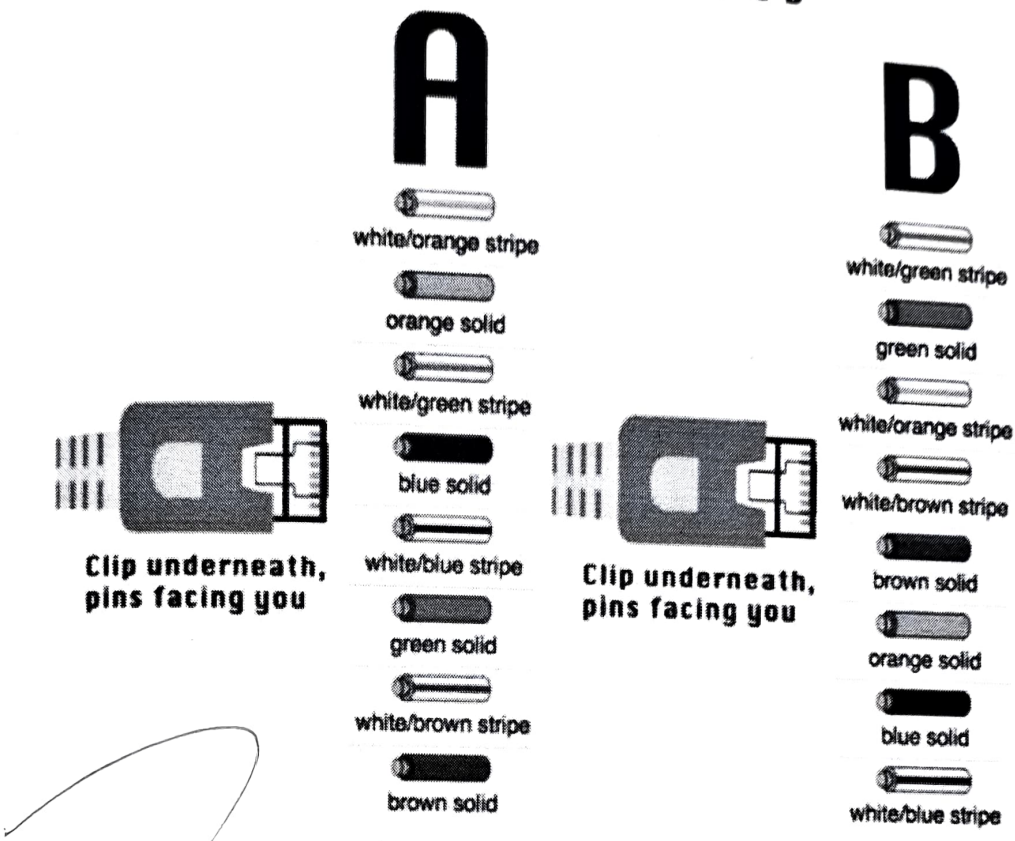


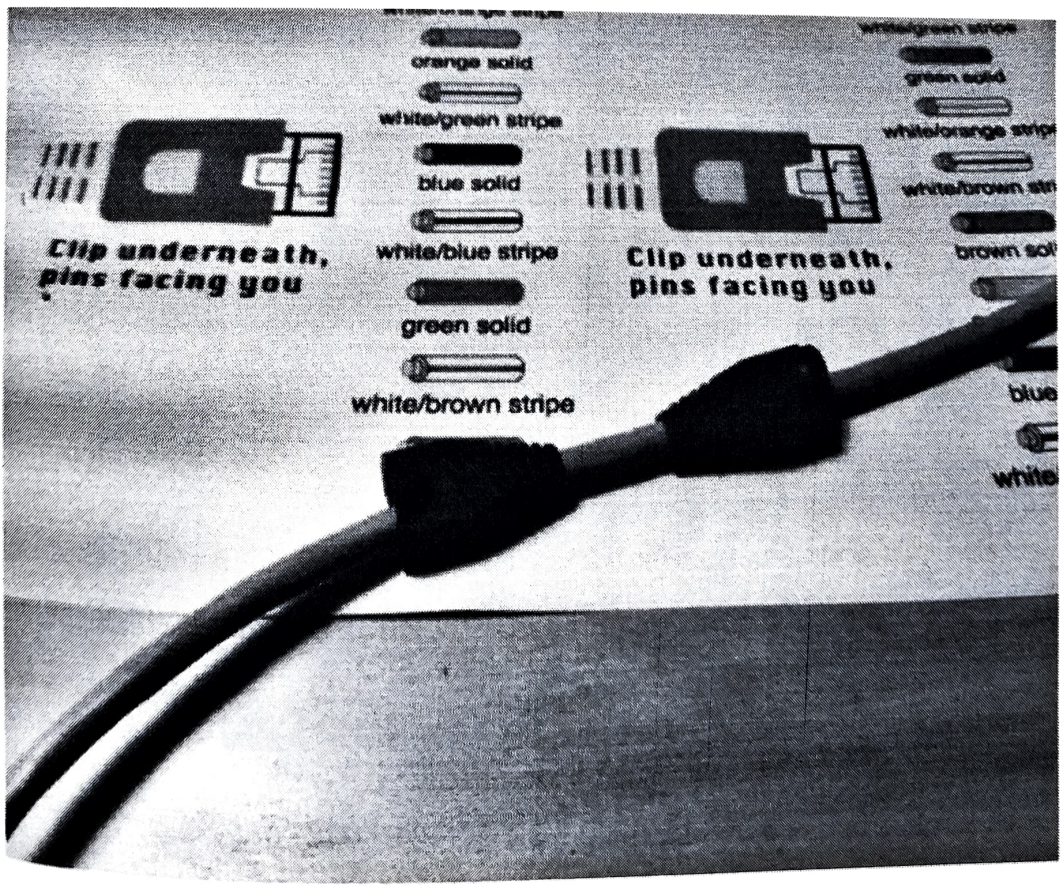




Difference between crossover cable and straight cable

straight through network cable: both sides should be A
Crossover cable: One side A, one side B





Step 4:

Arrange the wires in the correct order, trim them evenly, and insert them into the RJ 45 plug with the clip facing away and the gold pins, facing you.

Step 5:

push the cable fully into plug so the cable jacket is inside the connector. Trim and reinsert if needed.

Step 6:

place the RJ45 plug into the trimming tool and trim it firmly without applying excessive force.

Step 7:

Respect for the other end using B for a crossover cable or A for a straight through cable. Test the cable by connecting two devices.

Result:

Thus different types of network cables have studied.

Exp. 4 Setup and configure LAN using switch and ethernet cables

Date: 30/7/26

Aim.

Setup and configure a LAN (Local area network) using switch and ethernet cables.

What is a LAN?

A local area network refers to the network that connects devices within a limited area, such as an office building, school or home. It enables users to share resources, including data, printers and internet access. LAN connects devices to promote collaboration and transfer information between users, such as computers, servers, printers and switches.

How to setup a LAN

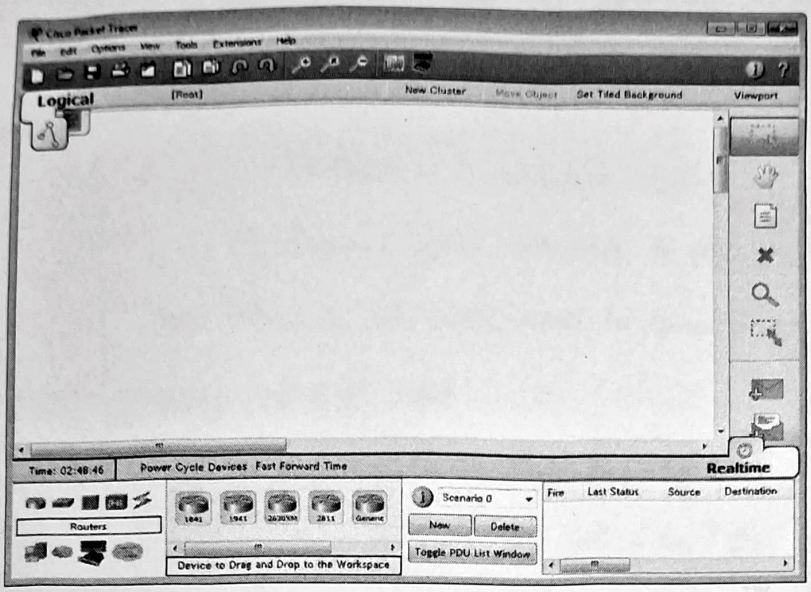
Step 1: plan and design an appropriate network topology taking into account network requirements and equipment location

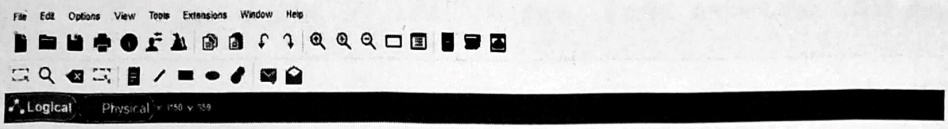
Step 2: you can take 4 computers, a switch with 8, 16, 24 ports which is sufficient for networks of these sizes, and 4 ethernet cables.

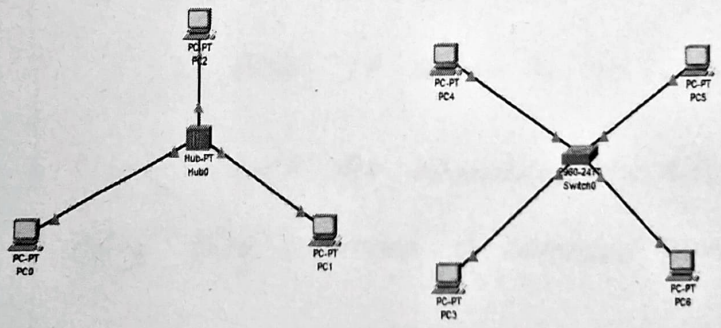
Step 3: Connect your computer to network switch via an ethernet cable, which is as simple as plugging one end of the ethernet cable into your computer and other end into your network switch.

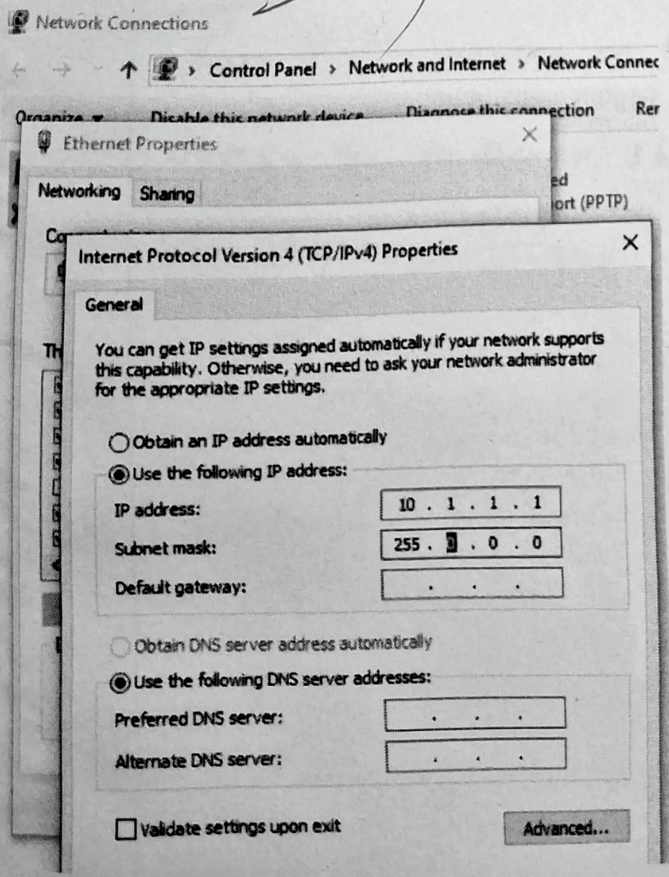
Step 4: Assign IP address to your PCs.

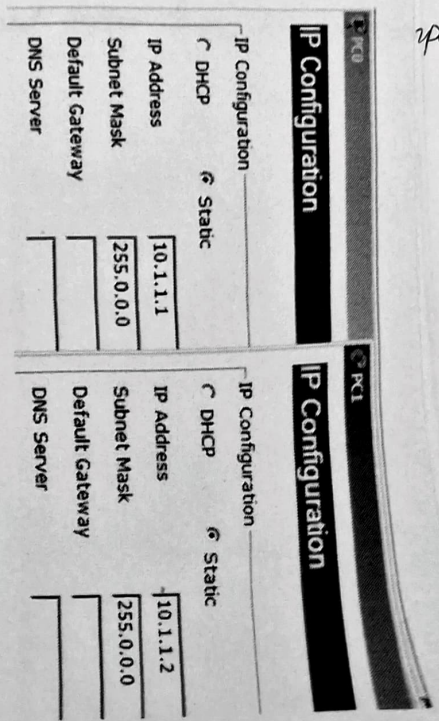
1) Log on to the client computer as administrator or owner.


Logical
[Root]
New Cluster
Move Object
Set Tiled Background
Viewport
Time: 02:48:46
Power Cycle Devices
Fast Forward Time
Realtime
Routers
Scenario 0
New
Delete
Toggle PDU List Window
Device to Drag and Drop to the Workspace
Fire
Last Status
Source
Destination


File Edit Options View Tools Extensions Window Help
Logical
Physical


PC-PT PC2
Hub-PT Hub0
PC-PT PC0
PC-PT PC1
PC-PT PC4
PC-PT PC5
2960-24TT Switch0
PC-PT PC3
PC-PT PC6


Network Connections
Control Panel › Network and Internet › Network Connec
Ethernet Properties
Networking
Sharing
Internet Protocol Version 4 (TCP/IPv4) Properties
General
You can get IP settings assigned automatically if your network supports this capability. Otherwise, you need to ask your network administrator for the appropriate IP settings.
Obtain an IP address automatically
Use the following IP address:
IP address: 10 . 1 . 1 . 1
Subnet mask: 255 . 0 . 0 . 0
Default gateway:
Obtain DNS server address automatically
Use the following DNS server addresses:
Preferred DNS server:
Alternate DNS server:
Validate settings upon exit
Advanced...


PC0
IP Configuration
IP Configuration
DHCP
Static
IP Address 10.1.1.1
Subnet Mask 255.0.0.0
Default Gateway
DNS Server
PC1
IP Configuration
IP Configuration
DHCP
Static
IP Address 10.1.1.2
Subnet Mask 255.0.0.0
Default Gateway
DNS Server

2) Click network and Internet connections.

3) Right click local Area Connection/Ethernet → Go to properties → select internet protocal (TCP/IPv4) → click on properties → select use the following ip addres option and assign ipaddress.

Step 5: Configure a network switch.

1) Connect your computer to the switch: To access the switch's web interface, you will need to connect your computer to the switch using ethernet cable.

2) Log into web Interface: open a web browser and enter ip addres of switch in the address bar. This should bring up login page for switch's web interface. Enter username and password to login.

3) Configure basic settings: Once you're logged in, you will be able to configure basic settings for switch.

4) Assign IP address as: 10.1.1.5, Subnet mask 255.0.0.0.

Step 6: Check the connectivity b/w switch and other machine by using ping command in command prompt of the device.

Step 7: select a folder → go to properties → click sharing tab → share it with everyone on the same LAN.

Step 8: Try to acces shared folder from others computer of the network.

Result:

A LAN was successfully setup using a switch and ethernet cable. Ip addresses were assigned to all computers and connectivity was verified using ping command and folder was succesfully shared and accessed b/w computers on same network.

Exp : 5

Date: 6/8/26

Packet capture using wireshark

Aim:

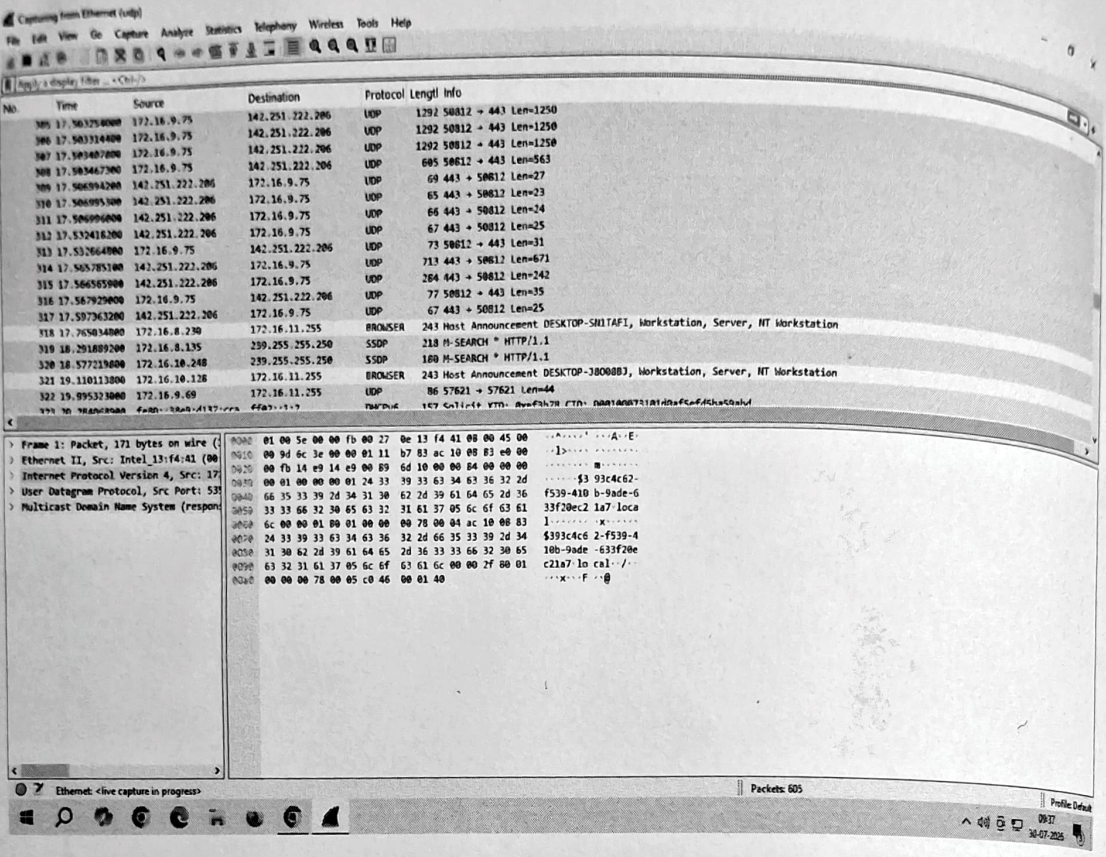
To capture filter inspect and analyze network protocols using wireshark packet analyzer.

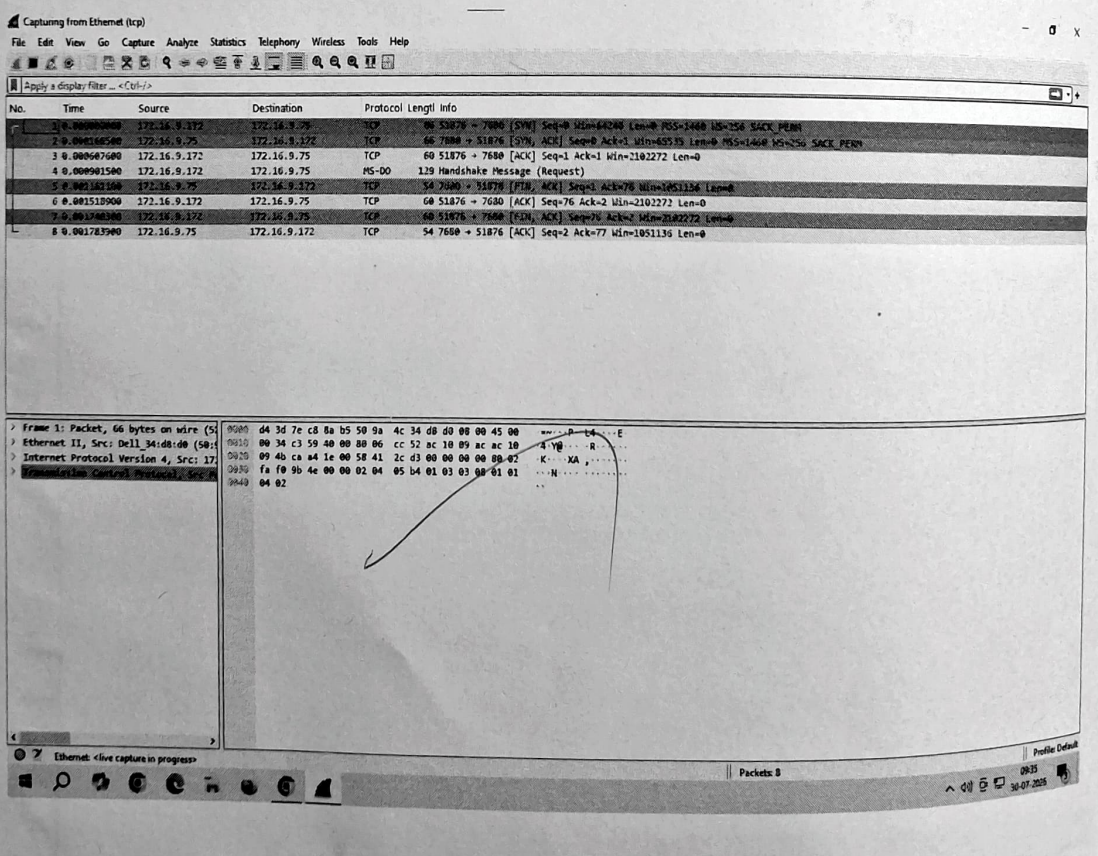
Algorithm:

1) Open wireshark.
2) Select the active network interface.
3) Configure capture to stop after 100 packets.
4) Start packet capture.
5) Apply display filter.
6) Inspect packet details and view the flow graph.
7) Save the captured packets.

Procedure:

* Launch wireshark and subset the active network interface.
* Go to capture → option and set stop capture after 100 packets.
* click start to capture packet.
    - TCP/UDP : tcp || udp
    - ARP : arp
    - DNS : dns
    - HTTP : http
* Save the capture file to future analysis.


Capturing from Ethernet (udp)
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
305 17.503258000 172.16.9.75 142.251.222.206 UDP 1292 50812 → 443 Len=1250
306 17.503314400 172.16.9.75 142.251.222.206 UDP 1292 50812 → 443 Len=1250
307 17.503407800 172.16.9.75 142.251.222.206 UDP 1292 50812 → 443 Len=1250
308 17.503467300 172.16.9.75 142.251.222.206 UDP 605 50812 → 443 Len=563
309 17.506094200 142.251.222.206 172.16.9.75 UDP 69 443 → 50812 Len=27
310 17.506095300 142.251.222.206 172.16.9.75 UDP 65 443 → 50812 Len=23
311 17.506096000 142.251.222.206 172.16.9.75 UDP 66 443 → 50812 Len=24
312 17.532418200 142.251.222.206 172.16.9.75 UDP 67 443 → 50812 Len=25
313 17.532664000 172.16.9.75 142.251.222.206 UDP 73 50812 → 443 Len=31
314 17.565785100 142.251.222.206 172.16.9.75 UDP 713 443 → 50812 Len=671
315 17.566585000 142.251.222.206 172.16.9.75 UDP 284 443 → 50812 Len=242
316 17.567929000 172.16.9.75 142.251.222.206 UDP 77 50812 → 443 Len=35
317 17.597363200 142.251.222.206 172.16.9.75 UDP 67 443 → 50812 Len=25
318 17.765834000 172.16.8.230 172.16.11.255 BROWSER 243 Host Announcement DESKTOP-SN1TAFI, Workstation, Server, NT Workstation
319 18.291889200 172.16.8.135 239.255.255.250 SSDP 218 M-SEARCH * HTTP/1.1
320 18.577219400 172.16.10.248 239.255.255.250 SSDP 180 M-SEARCH * HTTP/1.1
321 19.110113800 172.16.10.128 172.16.11.255 BROWSER 243 Host Announcement DESKTOP-J8O08BJ, Workstation, Server, NT Workstation
322 19.995323000 172.16.9.69 172.16.11.255 UDP 86 57621 → 57621 Len=44
Frame 1: Packet, 171 bytes on wire (
Ethernet II, Src: Intel_13:f4:41 (00
Internet Protocol Version 4, Src: 17
User Datagram Protocol, Src Port: 53
Multicast Domain Name System (respon
0000 01 00 5e 00 00 fb 00 27 0e 13 f4 41 08 00 45 00
0010 00 9d 6c 3e 00 00 01 11 b7 83 ac 10 08 83 e0 00
0020 00 fb 14 e9 14 e9 00 89 6d 10 00 00 84 00 00 00
0030 00 01 00 00 00 01 24 33 39 33 63 34 63 36 32 2d
0040 66 35 33 39 2d 34 31 30 62 2d 39 61 64 65 2d 36
0050 33 33 66 32 30 65 63 32 31 61 37 05 6c 6f 63 61
0060 6c 00 00 01 80 01 00 00 00 78 00 04 ac 10 08 83
0070 24 33 39 33 63 34 63 36 32 2d 66 35 33 39 2d 34
0080 31 30 62 2d 39 61 64 65 2d 36 33 33 66 32 30 65
0090 63 32 31 61 37 05 6c 6f 63 61 6c 00 00 2f 80 01
00a0 00 00 00 78 00 05 c0 46 00 01 40
Ethernet: <live capture in progress>
Packets: 605
Profile: Default
09:17
30-07-2025


Capturing from Ethernet (tcp)
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000000 172.16.9.172 172.16.9.75 TCP 66 51876 → 7680 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
2 0.000166500 172.16.9.75 172.16.9.172 TCP 66 7680 → 51876 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
3 0.000507600 172.16.9.172 172.16.9.75 TCP 60 51876 → 7680 [ACK] Seq=1 Ack=1 Win=2102272 Len=0
4 0.000901500 172.16.9.172 172.16.9.75 MS-DO 129 Handshake Message (Request)
5 0.001161100 172.16.9.75 172.16.9.172 TCP 54 7680 → 51876 [FIN, ACK] Seq=1 Ack=76 Win=1051136 Len=0
6 0.001518900 172.16.9.172 172.16.9.75 TCP 60 51876 → 7680 [ACK] Seq=76 Ack=2 Win=2102272 Len=0
7 0.001740300 172.16.9.172 172.16.9.75 TCP 60 51876 → 7680 [FIN, ACK] Seq=76 Ack=2 Win=2102272 Len=0
8 0.001783900 172.16.9.75 172.16.9.172 TCP 54 7680 → 51876 [ACK] Seq=2 Ack=77 Win=1051136 Len=0
Frame 1: Packet, 66 bytes on wire (5
Ethernet II, Src: Dell_34:d8:d0 (50:
Internet Protocol Version 4, Src: 17
Transmission Control Protocol, Src P
0000 d4 3d 7e c8 8a b5 50 9a 4c 34 d8 d0 08 00 45 00
0010 00 34 c3 59 40 00 80 06 cc 52 ac 10 09 ac ac 10
0020 09 4b ca a4 1e 00 58 41 2c d3 00 00 00 00 80 02
0030 fa f0 9b 4e 00 00 02 04 05 b4 01 03 03 08 01 01
0040 04 02
Ethernet: <live capture in progress>
Packets: 8
Profile: Default
09:35
30-07-2025

Result:

Successfully captured 100 network packets, applied display filter for TCP, UDP analyzed packet details using wireshark.

Exp: 6

Date: 13/8/24

Error Detection and Correction using Hamming Code

Aim:

To write a program to implement error detection and correction using the hamming code.

Algorithm:

1) Start
2) Read the text data from sender.
3) Convert each character of the text into its 8-bit ASCII binary form.
4) Calculate number of redundant bits required using hamming code.
5) Insert redundant bits at positions 1, 2, 4, 8...
6) Calculate the parity bits and generate hamming code.
7) Store the generated code in ~~hamming~~ channel file.
8) At receiver, read data from channel file.
9) Calculate the parity bits again to find error position.
10) If error position is non-zero, correct the bit at that position.
11) Remove the redundant bits.
12) Convert the remaining binary data into ASCII characters.
13) Display the corrected message.

Code:

```
def hamming(d):
    m = len(d); r = 0
    while 2**r < m + r + 1: r += 1
    a = ['0'] * (m + r); j = 0
    for i in range(1, len(a)+1):
        if i & (i-1): a[i-1] = d[j]; j += 1
    for p in [1 << p for p in range(r)]:
```

```
        for i in range (1, len(a)+1):
            if i & p : x^= int(a[i-1])
        a[p-1] = str(x)
    return ''.join(a)

s = input("Enter data : ")
b = ''.join(format(ord(c), '08b') for c in s)
c = hamming(b)
open("channel", "w").write(c)
print("Binary data:", b)
print("Channel data:", c)
c = open("channel").read()
e = int(input("Enter error position (0 for no error) : "))
if e: c = c[:e-1] + str(1-int(c[e-1])) + c[e:]
r=0
while 2**r < len(c)+1 : r+=1
x=0
for p in [1<<p for i in range(r)]:
    z=0
    for i in range(1, len(c)+1):
        if i & p : z^= int(c[x-1]) +c[x:]
d=''.join(c[i] for i in range(len(c)) if (i+1)&i)
print("Received data :", ''.join(chr(int(d[i:i+8],2))
for i in range(0, len(d), 8)))
```

Output:

```
Enter data: A
Binary data: 01000001
Channel data: 11010000101 1
Enter error position (0 for no error): 5
Error position: 5
Received data: A
```

Result:

Hamming code successfully detects the error position and corrects the erroneous bit, recovering original data.

Exp. 1

Date: 20/8/26

Flow Control at data link layer using sliding window protocol.

Aim:

To implement flow control at the Data link layer using the sliding window protocol and simulate the transmission of frames from one node to another.

Algorithm.

1) Start the program.

2) Set the total no. of frames and window size.

3) Sender sends frames equal to the current window size.

4) Receiver receives the frames and sends acknowledgement.

5) After receiving ACKs, sender slides the window forward.

6) Continue until all frames are transmitted.

7) Display the frame transmission and acknowledgement process.

8) Stop.

Code:

```
def sliding_window (total_frames, window_size):
    sender = 0
    print ("Sliding window protocol")
    while sender < total_frames:
        end = min (sender + window_size, total_frames)
        print ("In sender sends frames:")
        for frame in range (sender, end):
            print ("Frame", frame, "Sent")
```

```
        print("Receiver sends acknowledgements:")
        for frame in range(sender, end):
            print("ACK", frame, "received")
        sender = end
        if sender < total_frames:
            print("Window slides forward ...")
    print("\nAll frames transmitted successfully")

total_frames = int(input("Enter total no. of frames: "))
window_size = int(input("Enter window size: "))

sliding_window(total_frames, window_size)
```

Output:

```
Enter total number of frames: 8
Enter window size: 3
sliding window protocal

Sender sends frames:
Frame 0 sent
Frame 1 sent
Frame 2 sent

Receiver sends acknowledgement:
ACK 0 received
ACK 1 received
ACK 2 received.
Windows slides forward...

Sender sends frame:
Frame 3 sent
Frame 4 sent
Frame 5 sent
```

Receiver sends acknowledgement:

Ack 3 received

Ack 4 received

Ack 5 received

windows slides forward...

Sender sends frames

Frame 6 sent

Frame 7 sent

Receiver sends acknowledgements:

Ack 6 received

Ack 7 received

All frames transmitted successfully.

Result:

The sliding window protocol was successfully implemented. The program simulated the transmission of b/w sender and receiver using fixed-size window, with acknowledgement causing window to slide forward.

Exp: 8

Date:

Nmap Live Host Discovery using ARP, Icmp, Tcp and UDP

Aim:

To discover live hosts using Nmap scans such as ARP, Icmp, Tcp and UDP on the TryHackMe platform.

Algorithm:

1) Open TryHackMe platform and access the Nmap Live Host discovery room.

2) Study concept of live host discovery before performing network scanning.

3) Start with ARP scan to identify live host in local network.

4) Observe the ARP request and responses from target hosts

5) perform icmp scan to check whether target systems respond to Icmp requests.

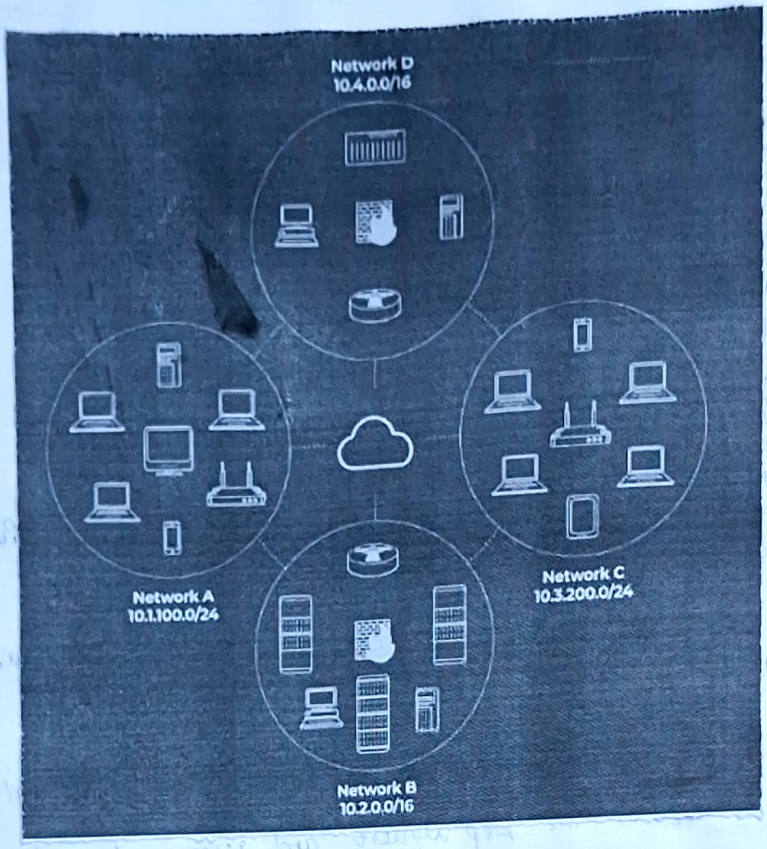
6) perform TCP SYN ping scan by sending TCP packets to required ports.

7) Perform TCP ACK ping scan and observe the response from target host.

8) perform UDP ping scan to determine whether the target system is reachable.

9) compare results obtained from ARP, ICMP, TCP, UDP host discovery methods.

10) Record the live hosts identified by Nmap and study the scan results.



Send Packet

From:

computer1

To:

computer1

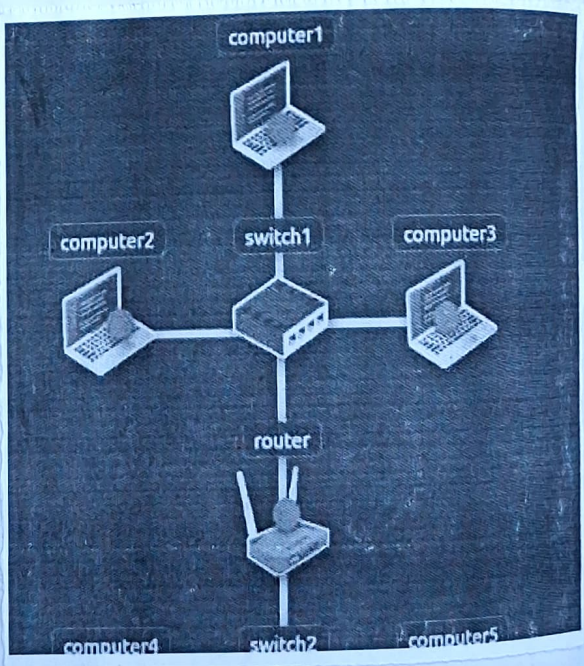
Packet Type:

arp_request

Data:

computer6

Send Packet



Send Packet

From:

computer4

To:

computer4

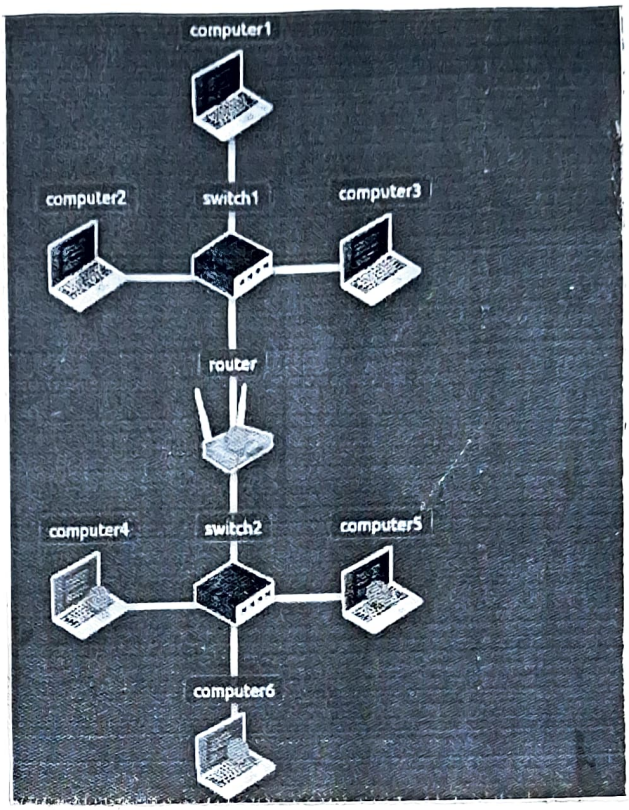
Packet Type:

arp_request

Data:

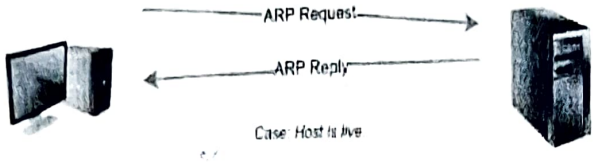
computer6

Send Packet



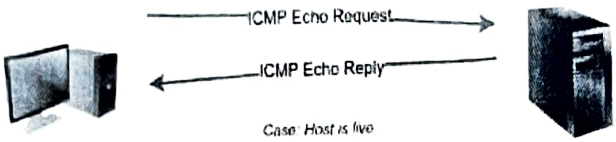
	ISO/OSI	TCP/IP	
7	Application Layer	Application Layer	HTTP, HTTPS, SMTP, POP3, IMAP, SSH, FTP, SNMP, Telnet, RDP,...
6	Presentation Layer		
5	Session Layer		
4	Transport Layer	Transport Layer	TCP, UDP
3	Network Layer	Network Layer	IPv4, IPv6, ICMP, IPsec
2	Data Link Layer	Link Layer	ARP, Ethernet (802.3), WiFi (802.11), DSL, Bluetooth,
1	Physical Layer		

nmap -PR -sn TARGET



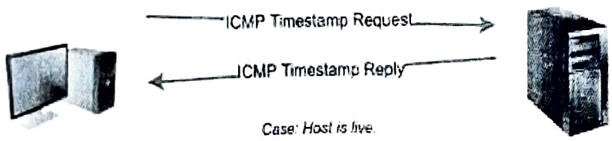
Case: Host is live

nmap -PE -sn TARGET



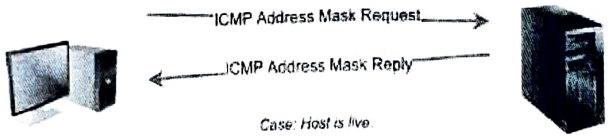
Case: Host is live

nmap -PP -sn TARGET



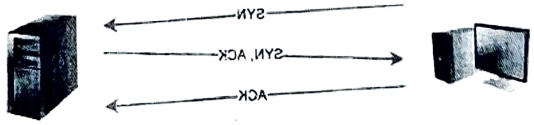
Case: Host is live.

nmap -PM -sn TARGET

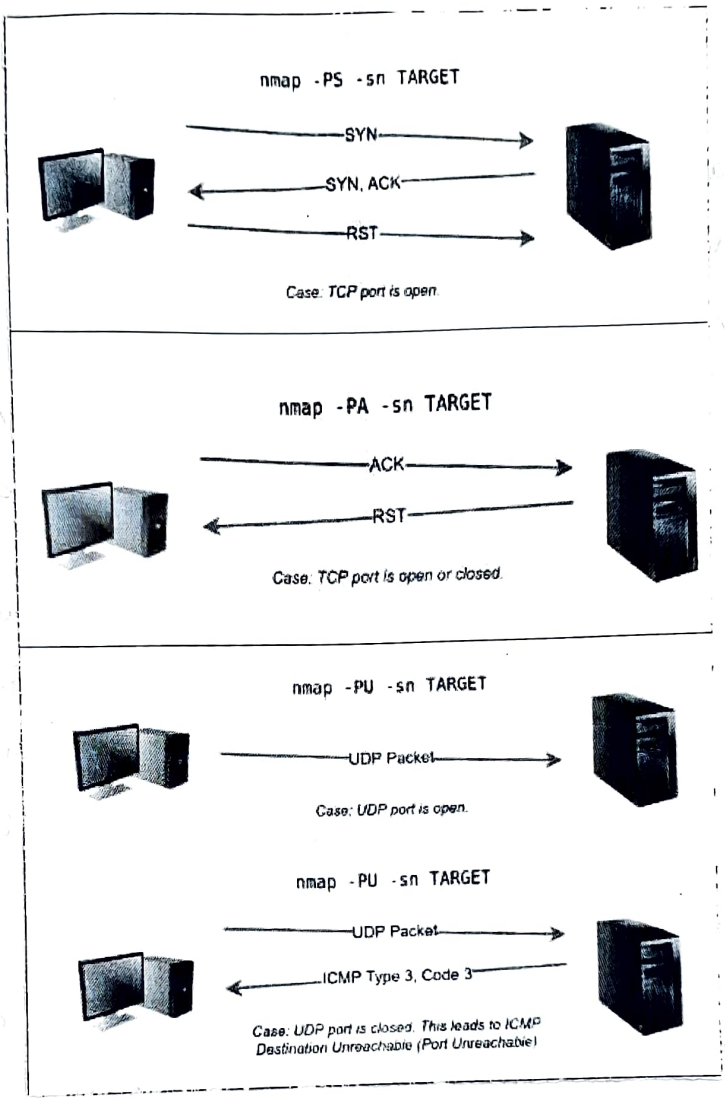


Case: Host is live.

TCP 3-Way Handshake



Case: TCP port is open



Result:

Thus, five hosts were successfully discovered using ARP, ICMP, TCP, UDP scanning techniques in Nmap.

EXP: 8(b)

Date:

Configuration of wireless LAN using Cisco packet tracker

Aim:

To design and configure a wireless LAN using cisco packet tracker with three pcs connected to a Linksys wireless router.

Algorithm:

1) Open Cisco tracker and create a new workspace.

2) Select a Linksys wireless router and place it in workspace.

3) Add three pcs to the network topology

4) Configure basic network settings of wireless router.

5) Configure the wireless network SSID as mothernetwork.

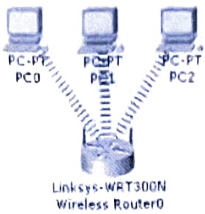
6) Configure WEP security for wireless network.

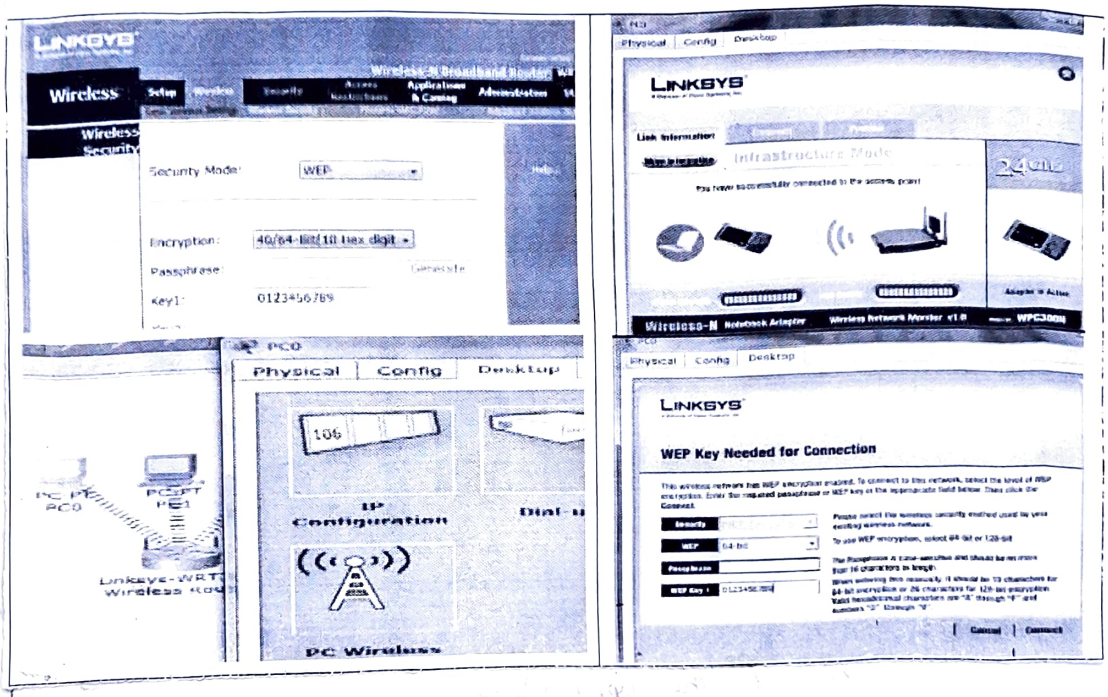
7) Open wireless configuration of first pc and select mothernetwork.

8) Enter configured security key and connect the first pc to the router.

9) Repeat the wireless configuration for second and third pcs.

10) Verify the wireless connection and test communication between the connected pcs.





Result:

Thus, wireless LAN was successfully designed and configured using cisco packet tracer with three PCs connected to the linksys wireless router.

Exp: 9

Date:

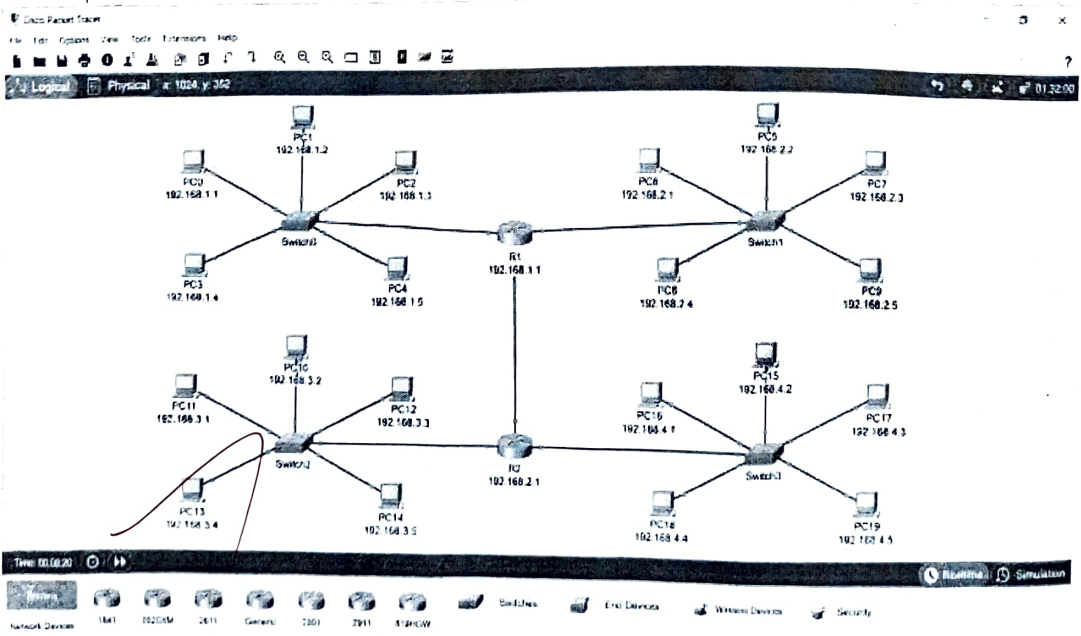
Subnetting Using Cisco Packet Tracer

Aim:

To design and configure a subnetted network using cisco packet tracer.

Algorithm:

1) Open cisco packet tracer.
2) Add a router, switch and pcs
3) connect the devices using suitable cables.
4) Open the router CLI
5) configure the router interfaces with ip addresses and subnet mask.
6) Enable the interfaces using no shutdown.
7) configure the ip addres and subnet mask of each pc.
8) Set the appropriate default gateway for each pc.
9) Use the ping command to test connectivity.



Result:

Thus, the subnetted network was successfully configured and connectivity was verified using ping.

Exp: 10 (a)

Date:

Internetworking with Router

Aim:

To design and configure an Internetwork using a router in cisco packet tracker.

algorithm:

1) Open cisco packet tracer.
2) Add one router and two pcs
3) connect pcs to the router.
4) configure the router Interfaces with ip addresses.
5) Enable Interfaces using no shutdown.
6) Configure ip address and subnet mask for pc 0.
7) Configure ip address and subnet mask for pc 1.
8) Set default gateways for both pcs.
9) Send PDU from pc 0 to pc 1.
10) Verify the communication between the two networks.

Result:

Thus, the Internetwork was succesfully configured using a router and connectivity was verified.

Exp: 10(b)

Date:

Wireless Internetworking

Aim:

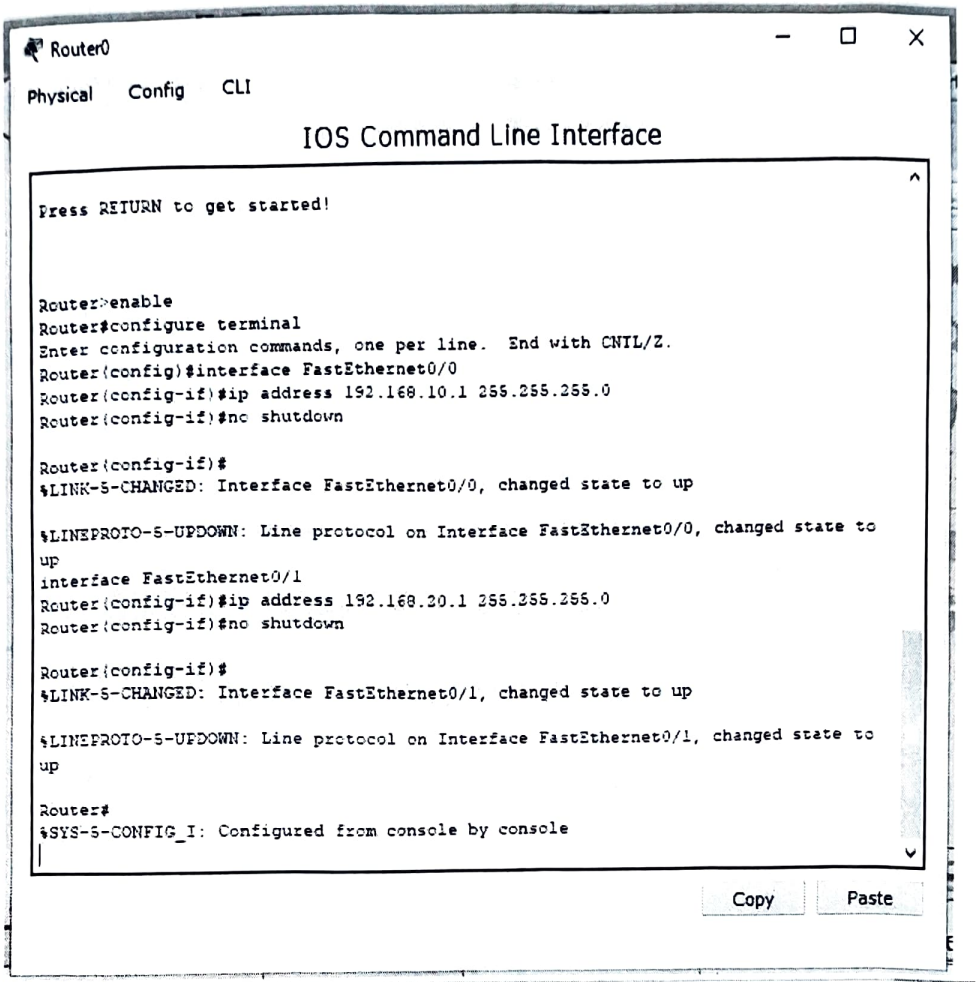
To design and configure an internetwork using a wireless router, DHCP server and Internet cloud.

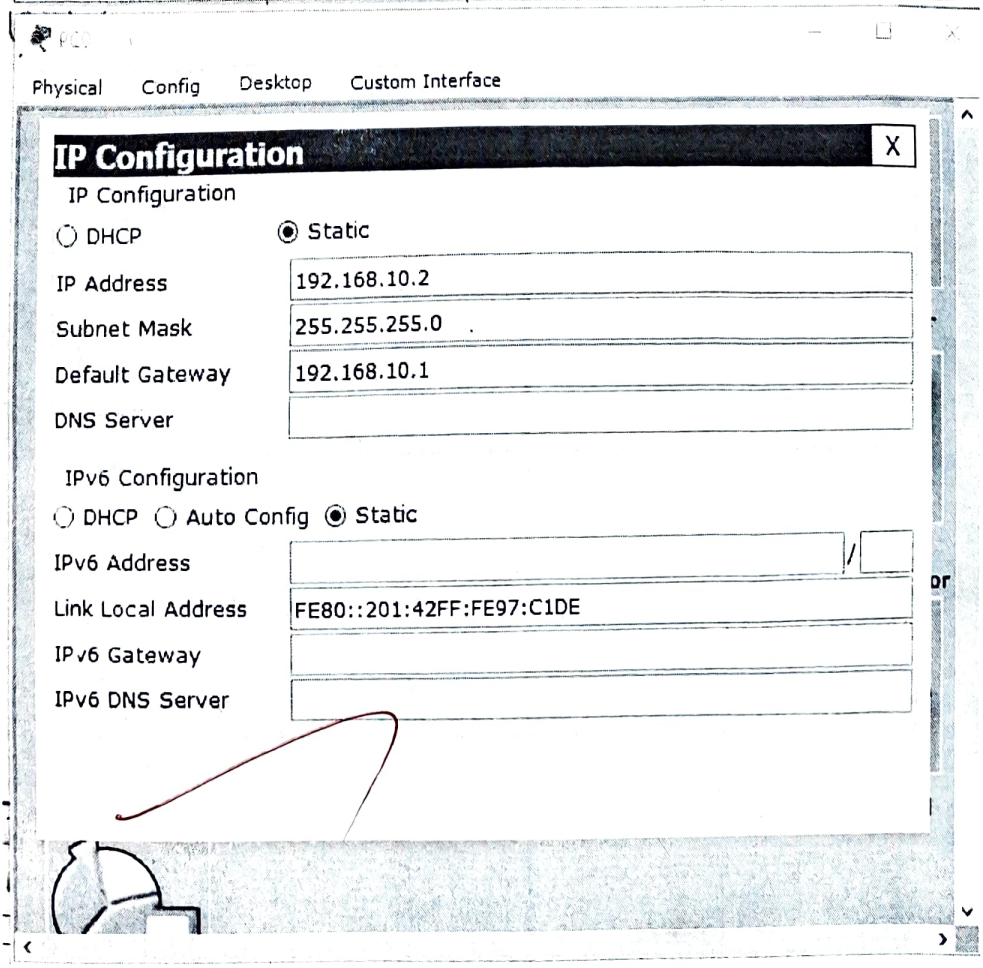
Algorithm:

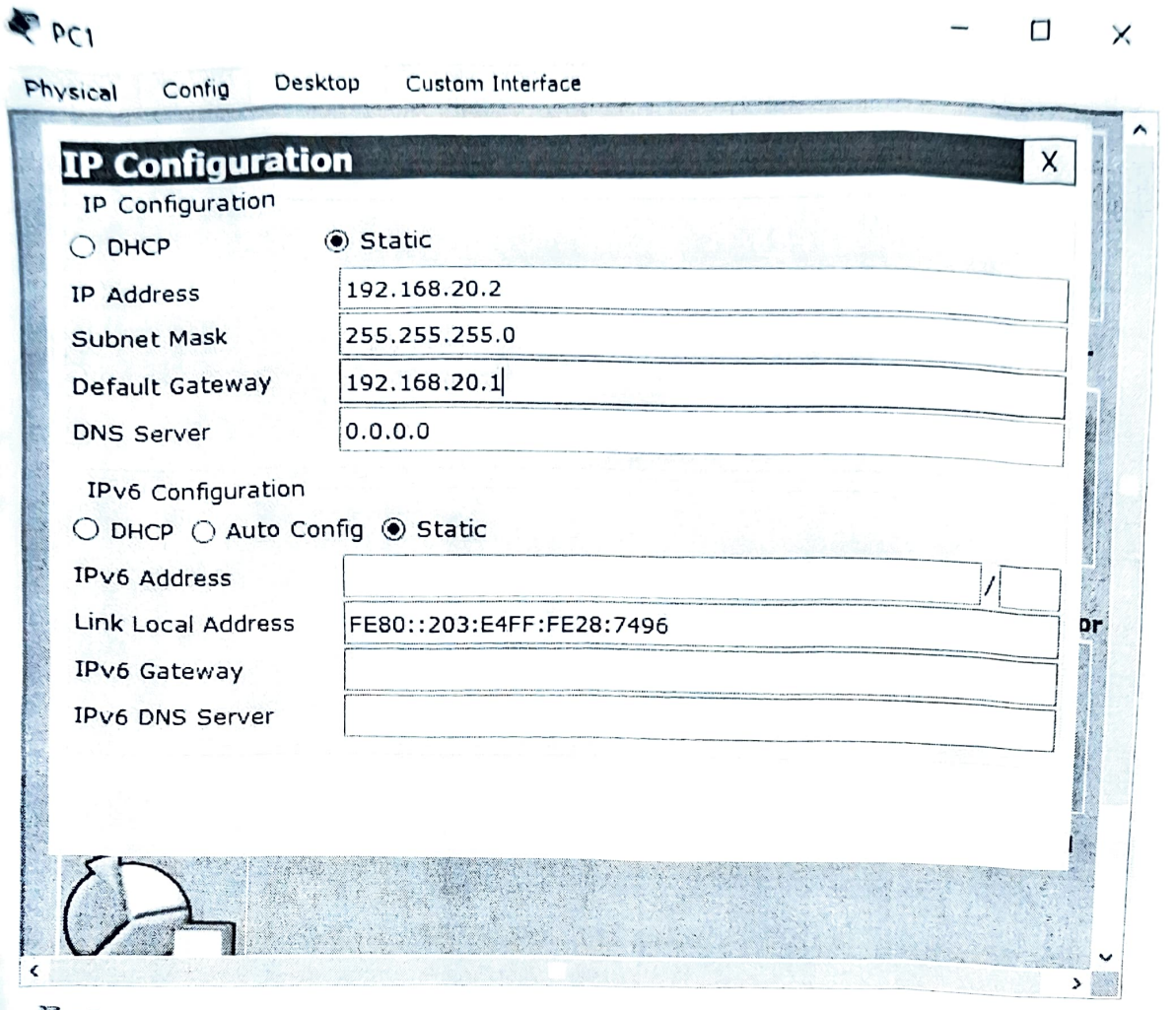
1) open cisco packet tracer.
2) Add a wireless router, DHCP server and Internet cloud.
3) Add the required wireless devices.
4) Connect the devices to the wireless router.
5) Configure the wireless router settings.
6) Configure DHCP server with IP settings
7) Enable DHCP for automatic IP assignment
8) Connect the wireless devices to the network.
9) Verify the assigned IP addresses.
10) Test connectivity between the devices and Internet cloud.

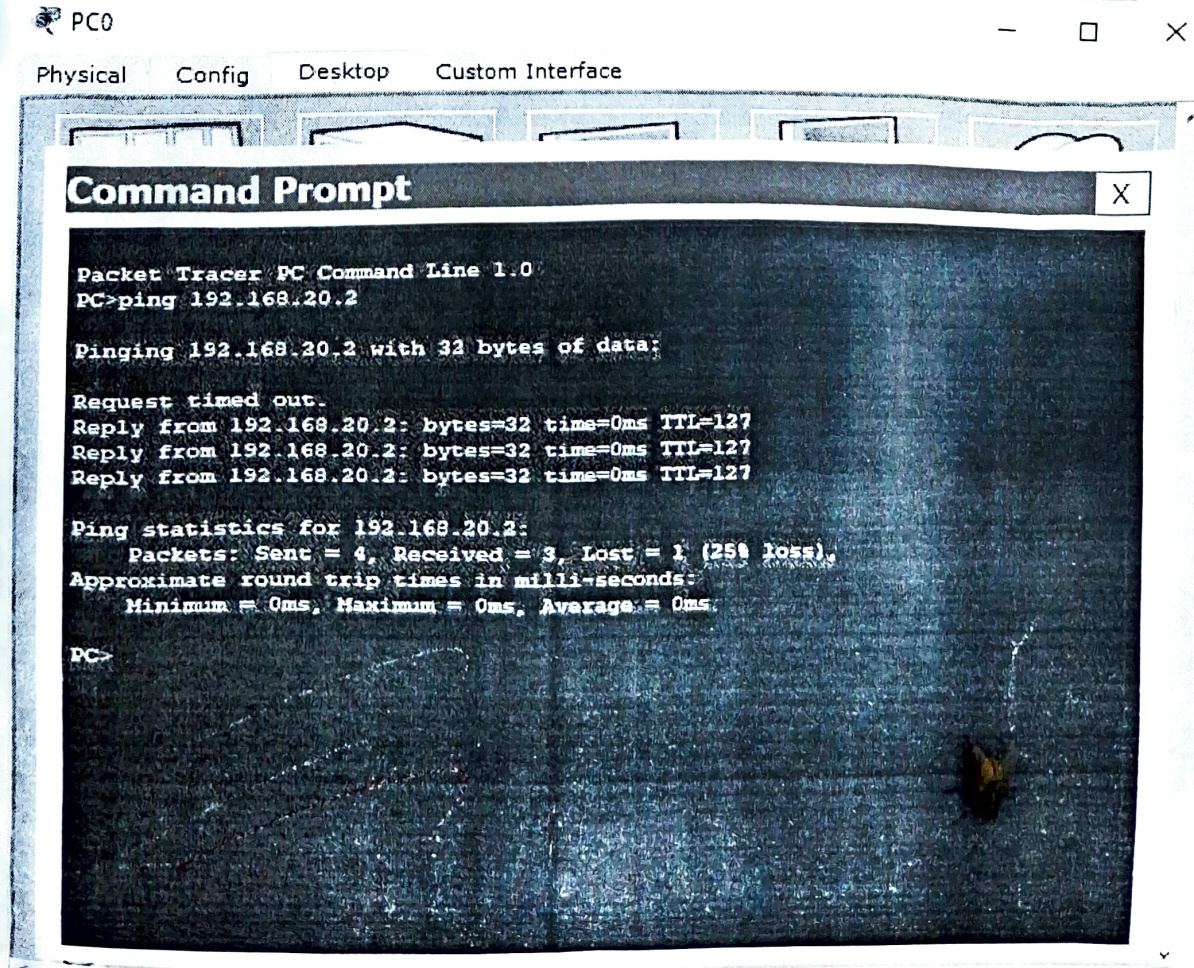
Result:

Thus, the wireless Internetwork was successfully configured and connectivity was verified.


Router0
Physical
Config
CLI
IOS Command Line Interface
Press RETURN to get started!
Router>enable
Router#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
Router(config)#interface FastEthernet0/0
Router(config-if)#ip address 192.168.10.1 255.255.255.0
Router(config-if)#no shutdown
Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/0, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/0, changed state to up
interface FastEthernet0/1
Router(config-if)#ip address 192.168.20.1 255.255.255.0
Router(config-if)#no shutdown
Router(config-if)#
%LINK-5-CHANGED: Interface FastEthernet0/1, changed state to up
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet0/1, changed state to up
Router#
%SYS-5-CONFIG_I: Configured from console by console
Copy
Paste


Physical
Config
Desktop
Custom Interface
IP Configuration
X
IP Configuration
DHCP
Static
IP Address
192.168.10.2
Subnet Mask
255.255.255.0
Default Gateway
192.168.10.1
DNS Server
IPv6 Configuration
DHCP
Auto Config
Static
IPv6 Address
Link Local Address
FE80::201:42FF:FE97:C1DE
IPv6 Gateway
IPv6 DNS Server


PC1
Physical
Config
Desktop
Custom Interface
IP Configuration
X
IP Configuration
DHCP
Static
IP Address
192.168.20.2
Subnet Mask
255.255.255.0
Default Gateway
192.168.20.1
DNS Server
0.0.0.0
IPv6 Configuration
DHCP
Auto Config
Static
IPv6 Address
Link Local Address
FE80::203:E4FF:FE28:7496
IPv6 Gateway
IPv6 DNS Server


PC0
Physical
Config
Desktop
Custom Interface
Command Prompt
X
Packet Tracer PC Command Line 1.0
PC>ping 192.168.20.2
Pinging 192.168.20.2 with 32 bytes of data:
Request timed out.
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Reply from 192.168.20.2: bytes=32 time=0ms TTL=127
Ping statistics for 192.168.20.2:
Packets: Sent = 4, Received = 3, Lost = 1 (25% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 0ms, Average = 0ms
PC>

Exp: 11(a)

Date.

Static Routing

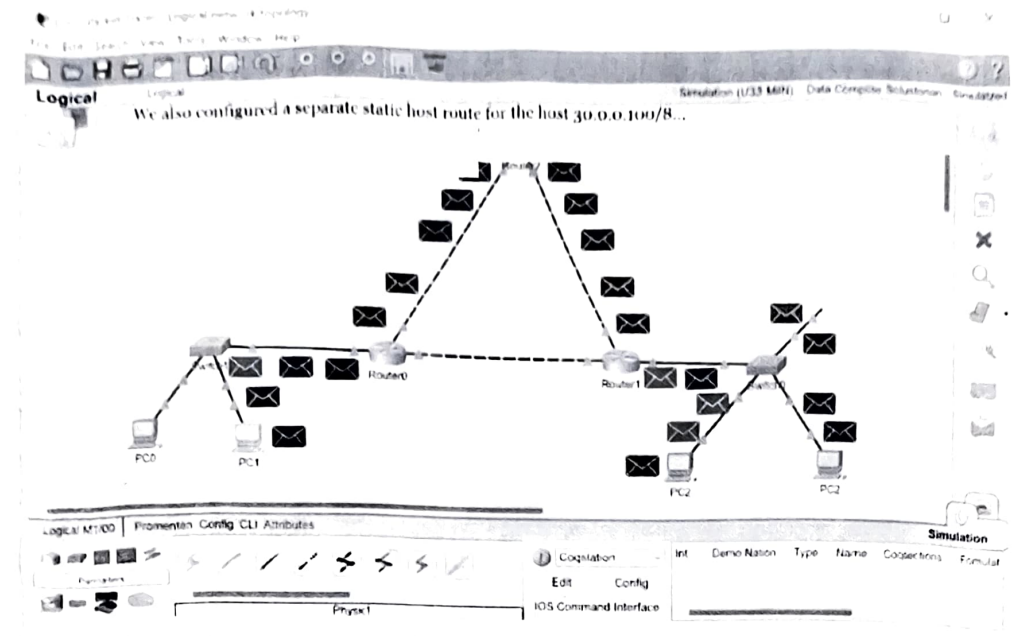
Aim:

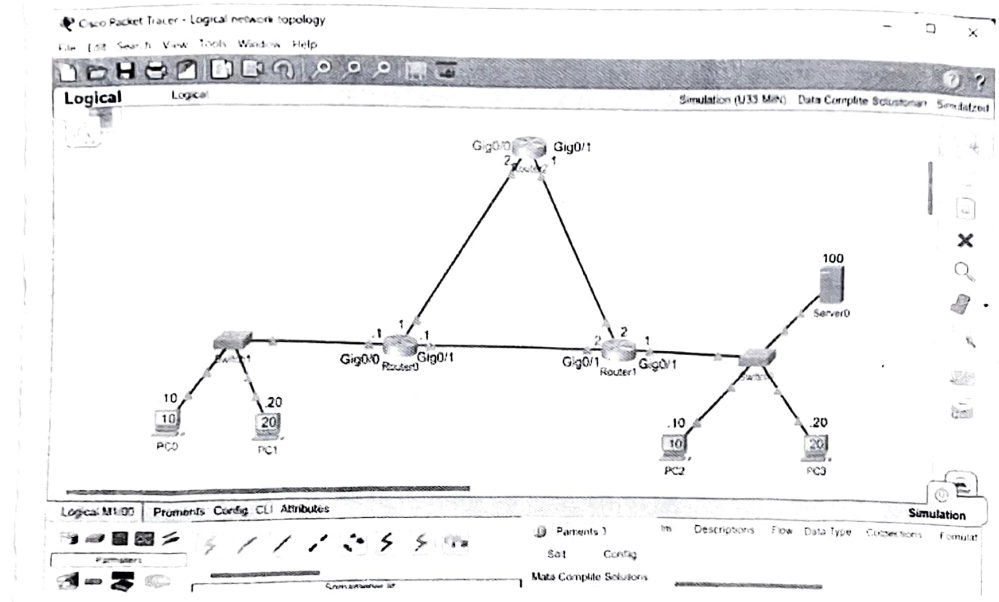
To configure static routing using cisco packet tracer.

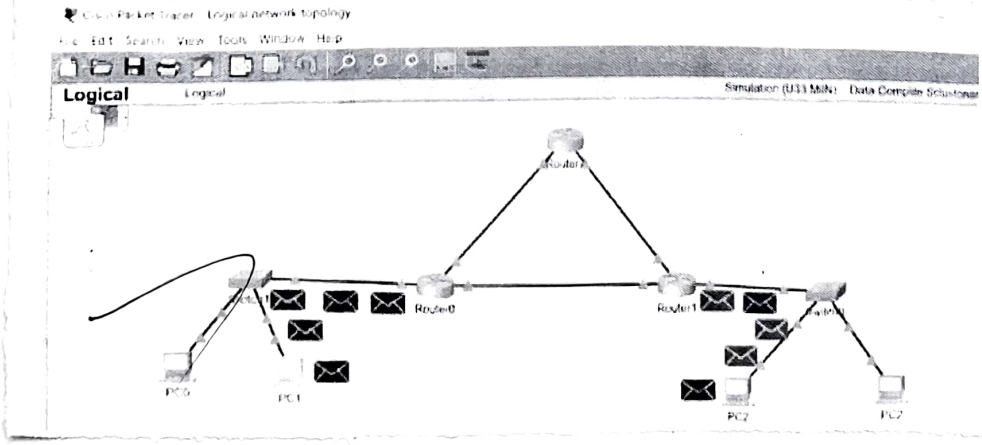
Algorithm:

1) Open cisco packet tracer.
2) Create required router and pc topology.
3) Connect the devices using suitable cables.
4) Assign ip addresses to pcs and router interfaces.
5) Enable router interface using no shutdown.
6) Configure static routes using ip route.
7) Configure the required backup route.
8) Check the routing table using show ip route.
9) Test connectivity using ping.
10) Verify communication between the networks.

We also configured a separate static host route for the host 30.0.0.100/8...







Result:

Thus, static routing was successfully configured and connectivity was verified.

Exp: 11(b)

Date:

RIP routing

Aim:

To configure RIP routing using cisco packet tracer.

Algorithm:

1) Open cisco packet tracer.
2) Create required router and pc topology.
3) Connect the device using suitable cables.
4) Assign ip address to pcs and router interfaces.
5) Enable router interfaces using no shutdown.
6) Enter RIP configuration mode using router rip.
7) Add the connected networks using network commands.
8) Configure RIP on the required routers.
9) Check the routing table.
10) Test connectivity using ping.

Results

Thus, RIP routing was succesfully configured and connectivity between networks was verified.

Exp: 12(a)

Date:

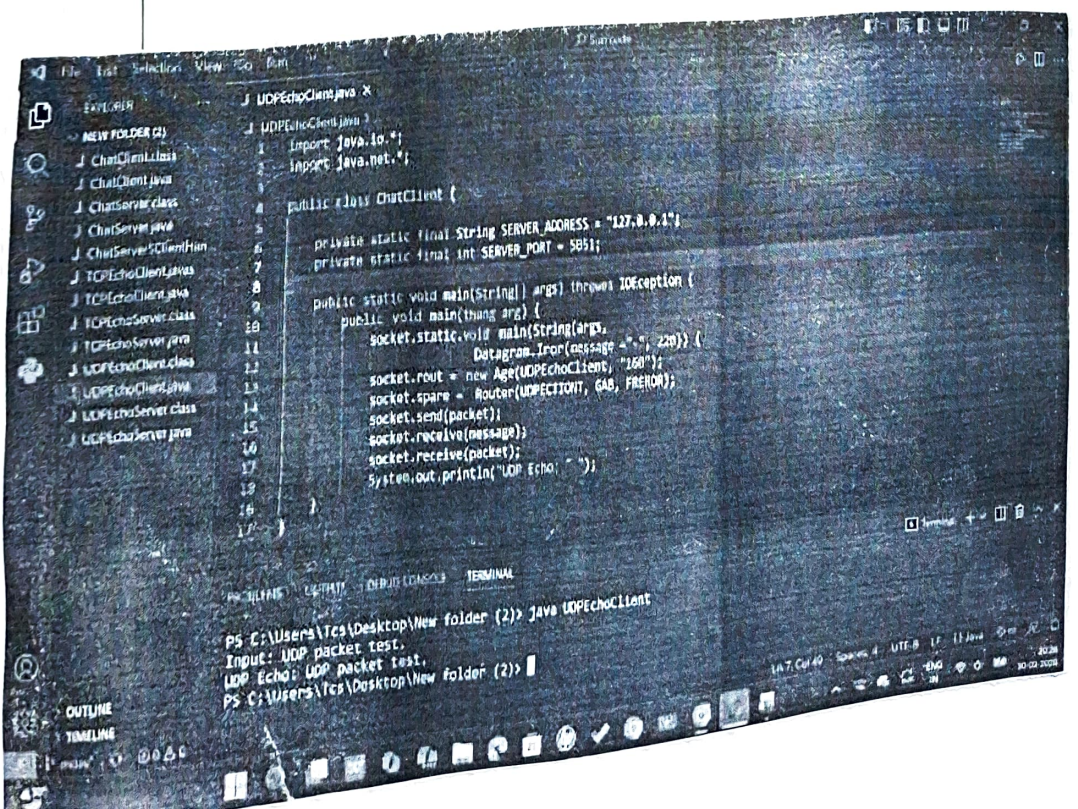
Echo client - server

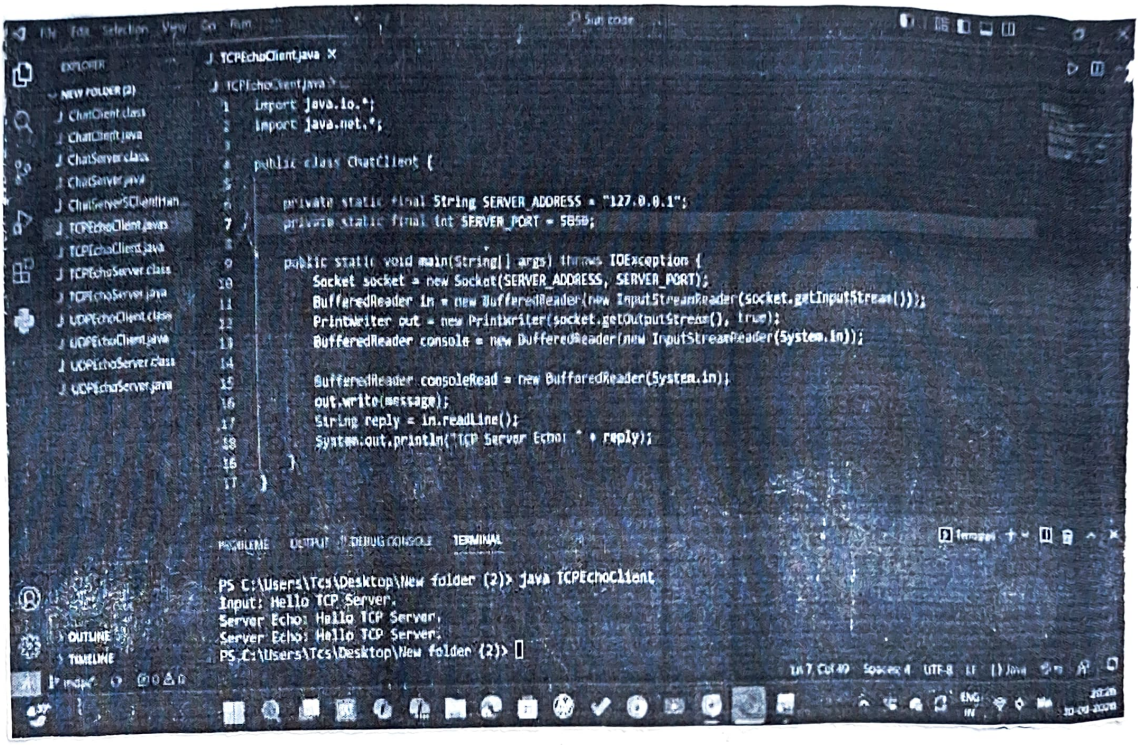
Aim:

To implement an echo client-server using TCP / UDP sockets.

Algorithm:

1) Create server socket.
2) Bind the socket to an ip addres and port.
3) Start listening for client requests.
4) Create client socket.
5) Connect the client to the server.
6) Enter a message from the client.
7) Send the message to the server.
8) Server receives and echoes the message.
9) Client receives the echoed message.
10) Close the sockets.





Aim:

Result:-

Thus, the echo client-server was successfully implemented.

Exp: 12(b)

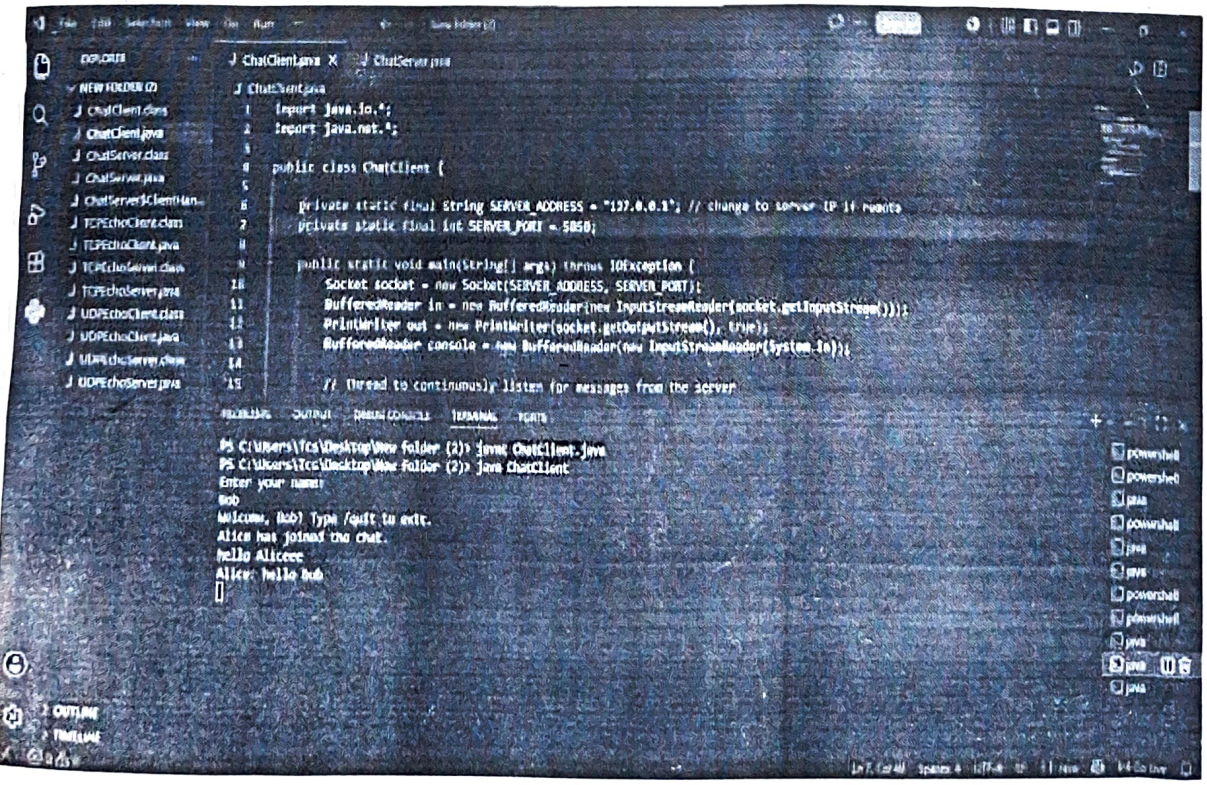
Chat client - server.

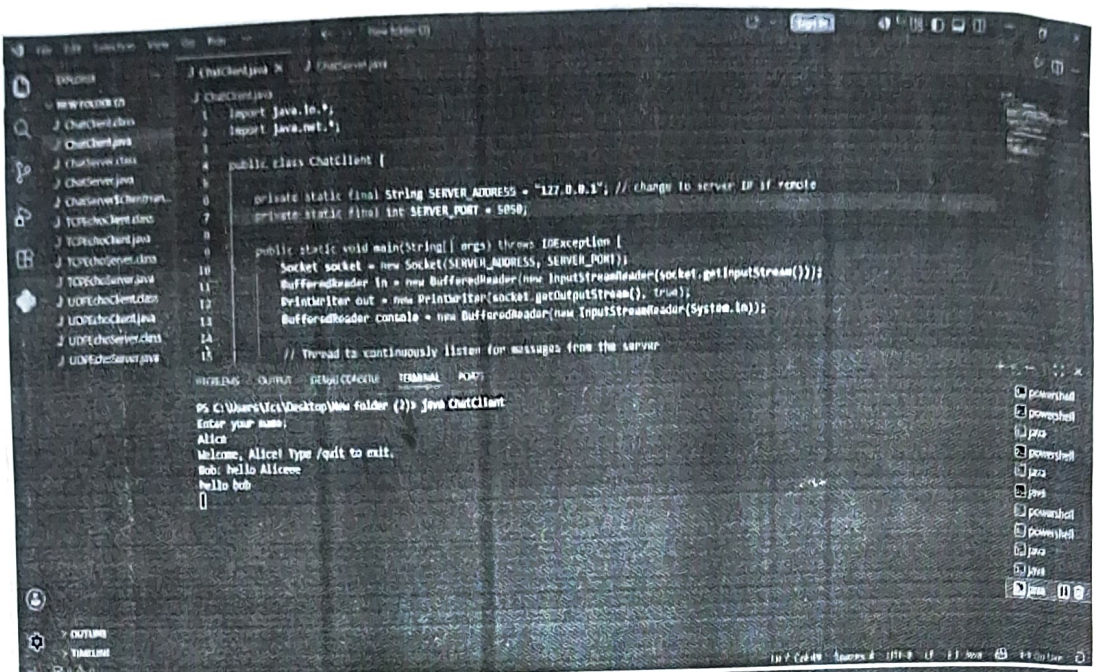
Aim:

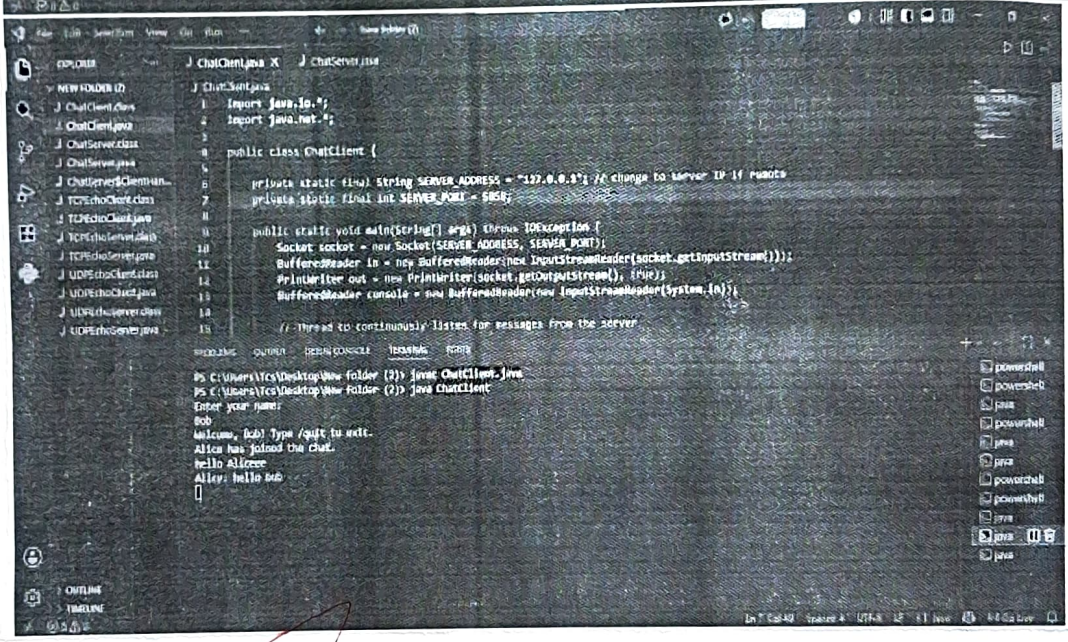
To Implement a chat program using socket programming.

Algorithm:

1) create the server p socket.
2) Bind the socket to an ip address and port.
3) Start listening for a client.
4) create client socket.
5) Establish connection between client and server.
6) Enter message from client.
7) Send and receive message between client and server.
8) Display the recived message.
9) continue communication until termination.
10) Close the sockets.







Results

Thus, the chat client-server was successfully implemented using socket programming.

Exp: 13

Date:

Implement your own ping program

Aim:

To Implement a ping program using python raw sockets to check reachability of a host and calculate the round-trip time.

program:

```
Import socket
Import struct
Import os
Import time
def checksum(data):
    total = 0
    for i in range(0, len(data)-1, 2):
        total += (data[i] << 8) + data[i+1]
    if len(data) % 2:
        total += data[-1] << 8
    total = (total >> 16) + (total & 0xffff)
    total += total >> 16
    return (~total) & 0xffff
def ping(host):
    try:
        dest_addr = socket.gethostbyname(host)
        sock = socket.socket(socket.AF_INET,
socket.SOCK_RAW, socket.IPPROTO_ICMP)
        packet_id = os.getpid() & 0xffff
        sequence = 1
        header = struct.pack("!BBHHH", 8, 0, 0,
                             packet_id, sequence)
        data = b"PING"
        checksum = checksum(header + data)
```

```
header = struct.pack("!BBHHH", 8, 0,
                     checksum, packet_id, sequence)
packet = header + data
start = time.time()
sock.sendto(packet, (dest_addr, 1))
sock.settimeout(2)
sock.recvfrom(1024)
end = time.time()
rtt = (end - start) * 1000
print("Reply from", dest_addr, ": time = %.2f ms" % rtt)
sock.close()
except socket.timeout:
    print("Request timed out")
except permissionError:
    print("permission denied. Run the program as administrator / root.")
host = input("Enter destination host: ")
ping(host)
```

Sample Input:

Enter destination host: 8.8.8.8

Output:

Reply from 8.8.8.8 : time = 24.53 ms

Result:

Thus, a ping program was implemented successfully using socket programming to determine host reachability and calculate the round-trip time.

Ex: 4
Date:

Packet Sniffing Using Raw Sockets

Aim:

To implement a packet sniffing program using python raw sockets and display the source IP, destination IP, protocal of captured packets.

program:

```
import socket
import struct
sock = socket.socket( socket.AF_INET, socket.SOCK_
                      RAW, socket.IPPROTO_IP )
print ( " packet sniffer started ... " )
print( " Waiting for packets ... \n " )
while True:
    packet, addrem = sock.recvfrom( 65535 )
    ip_header = packet [0:20]
    iph = struct.unpack ( "! BBHHHBBH4s 4s ",
          ip_header )
    source_ip = socket.inet_ntoa ( iph [8] )
    destination_ip = socket.inet_ntoa ( iph [9] )
    protocal = iph [6]
print ( " ---------- " )
    print ( "Source IP : ", source_ip )
    print ( "Destination IP : ", destination_ip )
    print ( " protocal : ", protocal )
    print ( " packet size : ", len(packet), "bytes " )
print ( " ---------- " )
```

Output:

```
Packet Sniffer started ....
Waiting for packets...
-------------------------
Source Ip      : 192.168.1.10
Destination Ip : 142.250.183.14
protocal       : 6
Packet size    : 60 bytes
-------------------------
-------------------------
Source Ip      : 8.8.8.8
Destination Ip : 192.168.1.10
protocal       : 1
Packet size    : 84 bytes
-------------------------
```

Result:

Thus, the ping program and packet sniffing program were successfully implemented using python raw sockets.

Analysis of various Types of servers using Webalizer.

Aim:

To analyze the web server log files using the webalizer tool and study various server statistics such as number of requests, visitors, transferred data, URLs and response status codes.

Algorithm:

1) Start webalizer application.
2) Install and configure webalizer on the system.
3) Locate web server access log file.
4) Configure webalizer with required log-file path and output directory.
5) Run webalizer on selected log file.
6) Webalizer reads and analyzes server log entries.
7) It generates statistics such as:
   - Number of visits.
   - No. of requests.
   - pages accessed
   - Data transferred.
   - HTTP response codes
   - Referring sites.
8) Open the generated HTML report in web browser.
9) Analyze server activity using generated graphs and tables.
10) Repeat analysis for different server log files if required.
11) Compare obtained server statistics.
12) Record the observations and terminate the experiment.

Input:

Web server access log files containing information such as:

- client IP
- Date and time
- Requested URL
- HTTP method
- HTTP transfer code
- Referer
- User agent.

access.log

Output:

Webalizer generates an HTML-based statistical report containing:

Webalizer statistics

Total requests	:	1250
Total visits	:	480
Total pages	:	920
Total Bytes	:	15.4 MB
Successful Requests	:	1180
404 Errors	:	45

Result:

Thus, webalizer tool was used successfully to analyze web server log files, and various server statistics were obtained.